← CS 180 · Home

CS 180 · Project 3 · Parts A & B

Fun with Flow Models

Painting images out of noise with a pretrained text-to-image flow model, then training one from scratch on MNIST.

Part A 1.1 1.2 1.3 1.4 1.5 1.6 2.1 3.1 3.2 3.3 3.4

Part B 1.1 1.2 2.1 2.2 2.3 2.4 2.5 2.6

Part A is just exploring different aspects of flow matching, using a pretrained text-to-image flow model (PixNerd).

model **PixNerd-XXL-P16-T2I** · images **512 × 512** · seed **180** · Euler step **h = 0.02** · CFG weight **w = 7** · prompt **"a high quality photo"**

$$x_t = (1-t)\,\epsilon + t\,x_1, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad t = 0 \text{ is noise, } t = 1 \text{ is clean}$$

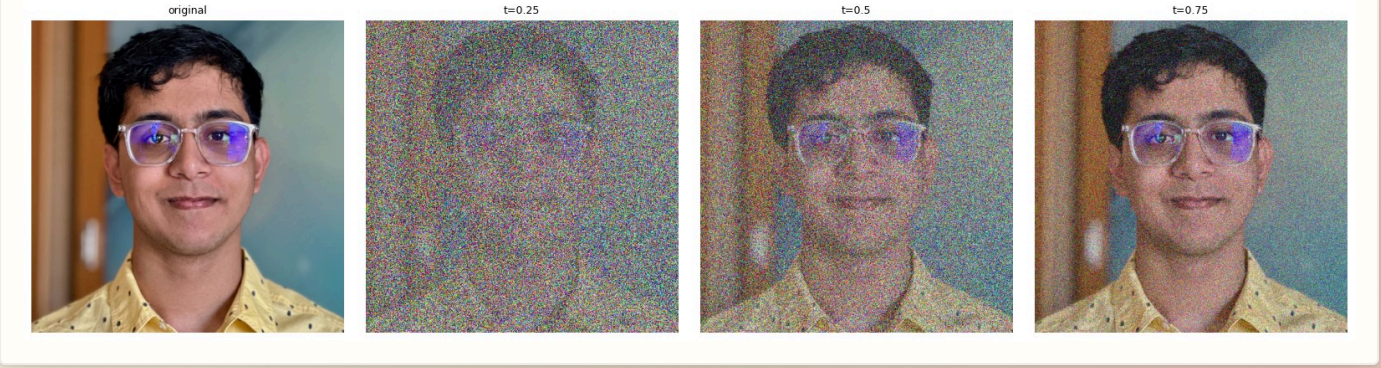
Section 1

Visualizing Training & Sampling

1.1 Constructing Noisy Training Inputs

$$x_t = (1 - t)\,\epsilon + t\,x_1$$

To make a noisy input I mix my image x_1 with Gaussian noise ϵ along a straight line, $x_t = (1 - t)\epsilon + tx_1$, using the same noise sample for every t. At t = 0.25 you can barely see my face, yet at t = 0.75 my face is mostly visible.

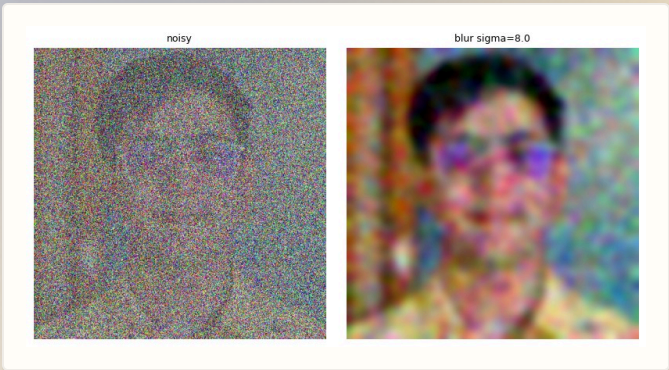


original, then t = 0.25 · 0.50 · 0.75 with the same noise sample

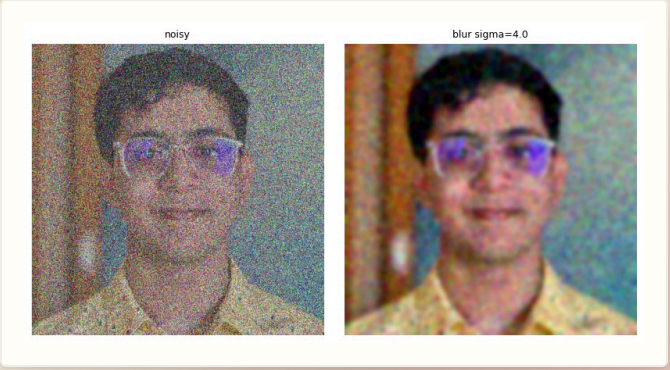
1.2 Classical Denoising

$$\hat{x}_1 = G_\sigma * \frac{x_t}{t}, \qquad \sigma \in \{1, 2, 4, 8, 16\} \text{ chosen by lowest MSE}$$

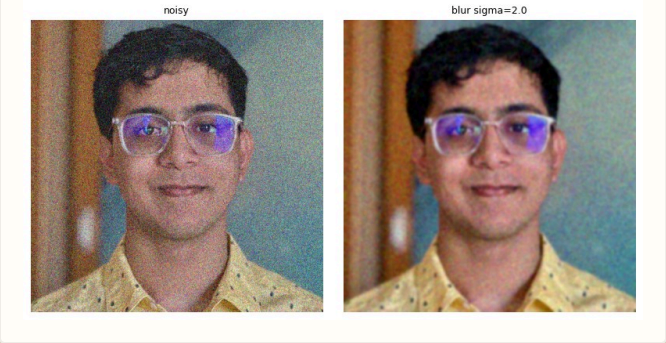
As a classical baseline I divide x_t by t to undo the scaling, then Gaussian blur it, trying σ ∈ {1, 2, 4, 8, 16} and keeping the one with the lowest MSE. At t = 0.25 it's still blurry and you can't see much (big colored blotches), while at t = 0.75 the blur basically clears the noise. The best σ decreases because with less noise, the low-pass filter can use a smaller and smaller σ to get rid of unwanted high-frequency things like noise (since it's less apparent). With more noise, you need more blur to try to remove it.



t = 0.25 · best σ = 8



t = 0.50 · best σ = 4

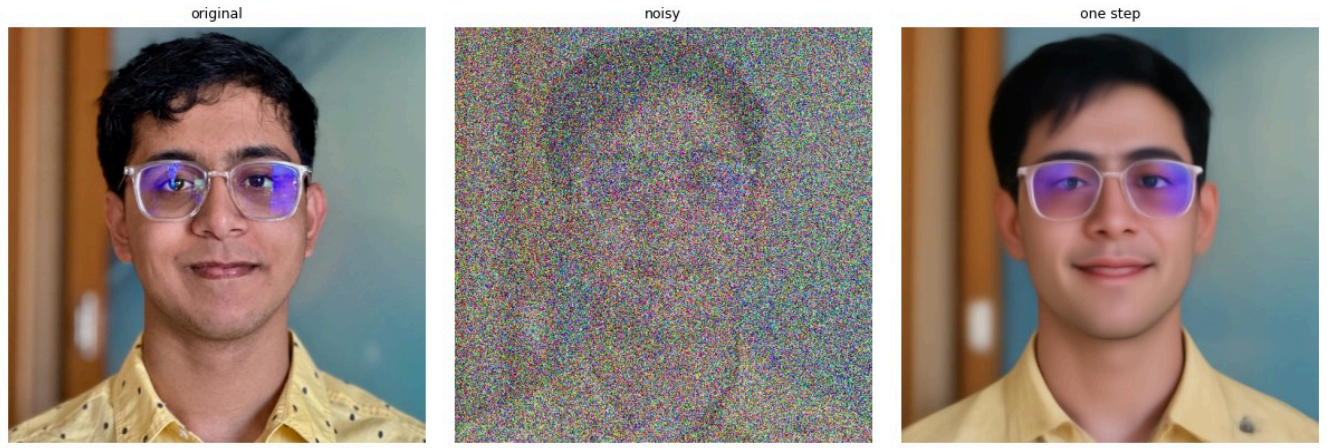


t = 0.75 · best σ = 2

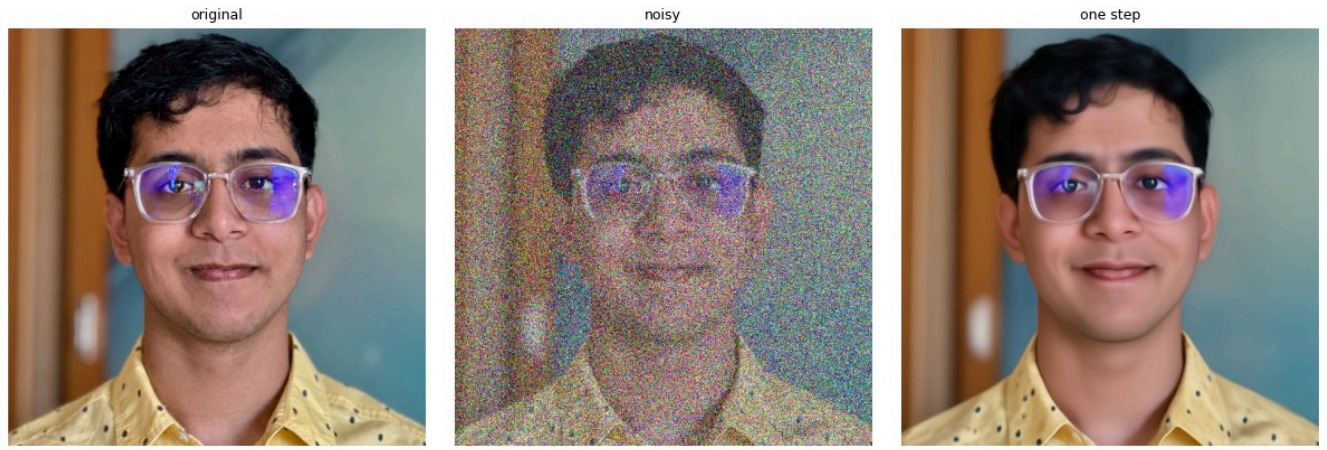
1.3 One-Step Denoising

$$\hat{x}_1 = x_t + (1 - t)\, v_\theta(x_t, t, c)$$

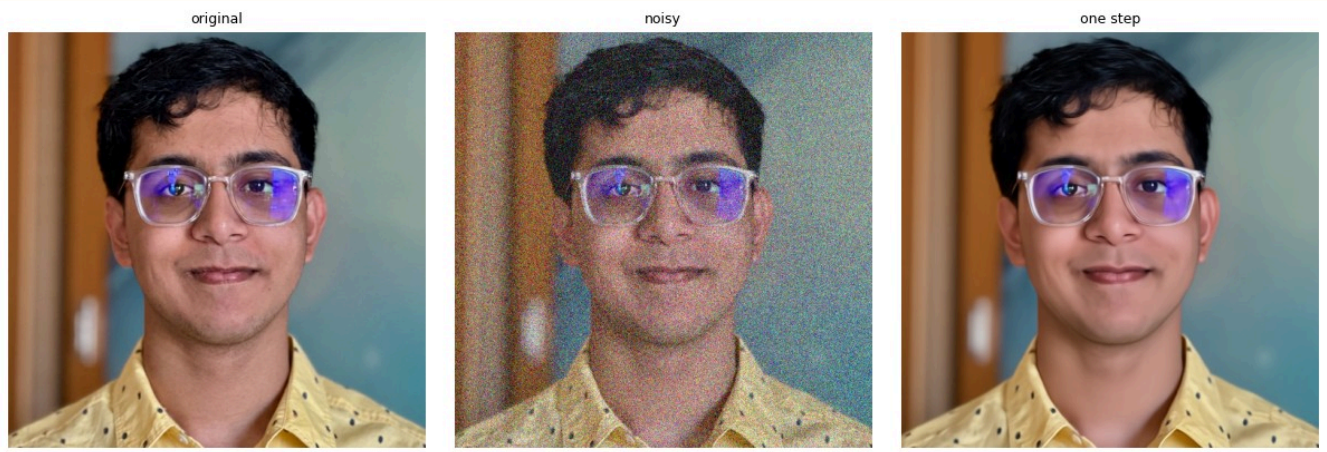
Here the model predicts the velocity once, and I jump straight to the end with $\hat{x}_1 = x_t + (1-t)v_\theta$. At t = 0.25 it doesn't look like me, while at t = 0.75 it's pretty close to me (t = 0.5 also gets somewhat close).



original · noisy · one step at t = 0.25



original · noisy · one step at t = 0.50



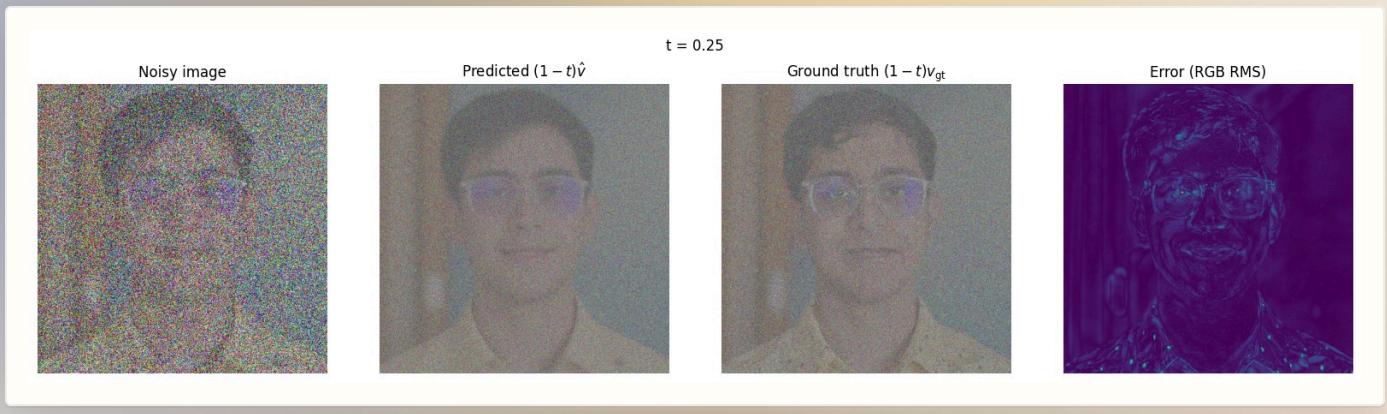
original · noisy · one step at $t = 0.75$

1.4 Visualizing Velocity

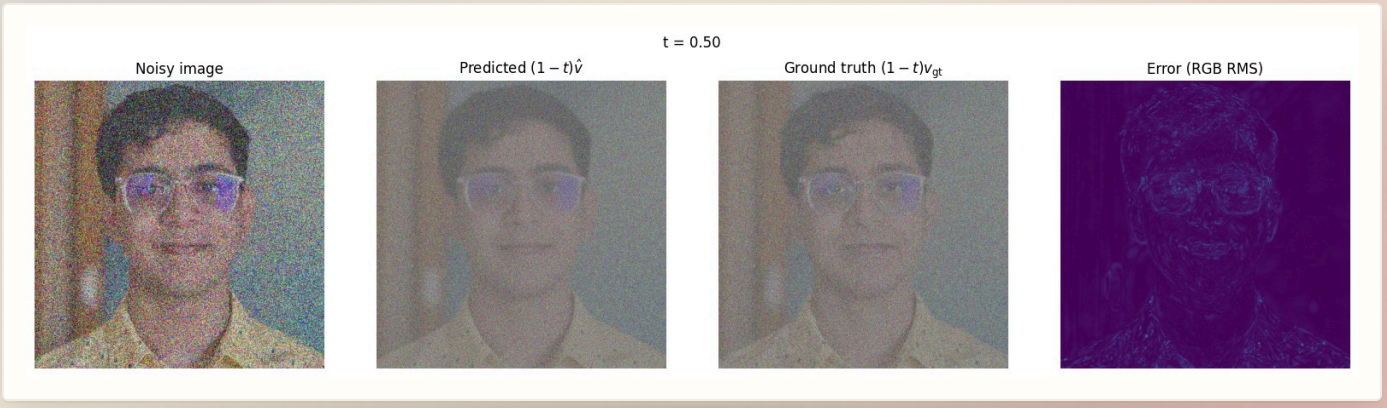
$$v_{gt} = x_1 - \epsilon, \qquad (1-t)\, v_{gt} = x_1 - x_t, \qquad E_t = \sqrt{\text{mean}_{RGB}\left[(1-t)(\hat{v}_t - v_{gt})\right.}$$

Each row shows the noisy image, the predicted update $(1-t)\hat{v}$, the ground-truth update $(1-t)v_{gt} = x_1 - x_t$, and the per-pixel RGB RMS error between them. At t = 0.25 the error mostly shows the edges and features, so most of the error lies in the face. As we get to t = 0.75 the error map is basically black and shows little error.

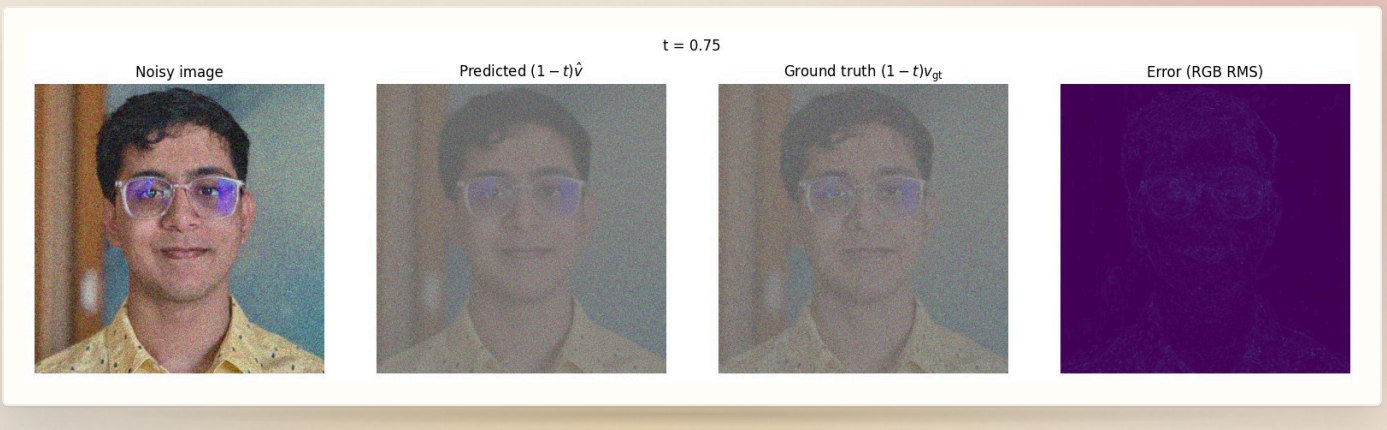
the original image



noisy · predicted update · ground-truth update · error at t = 0.25



noisy · predicted update · ground-truth update · error at t = 0.50

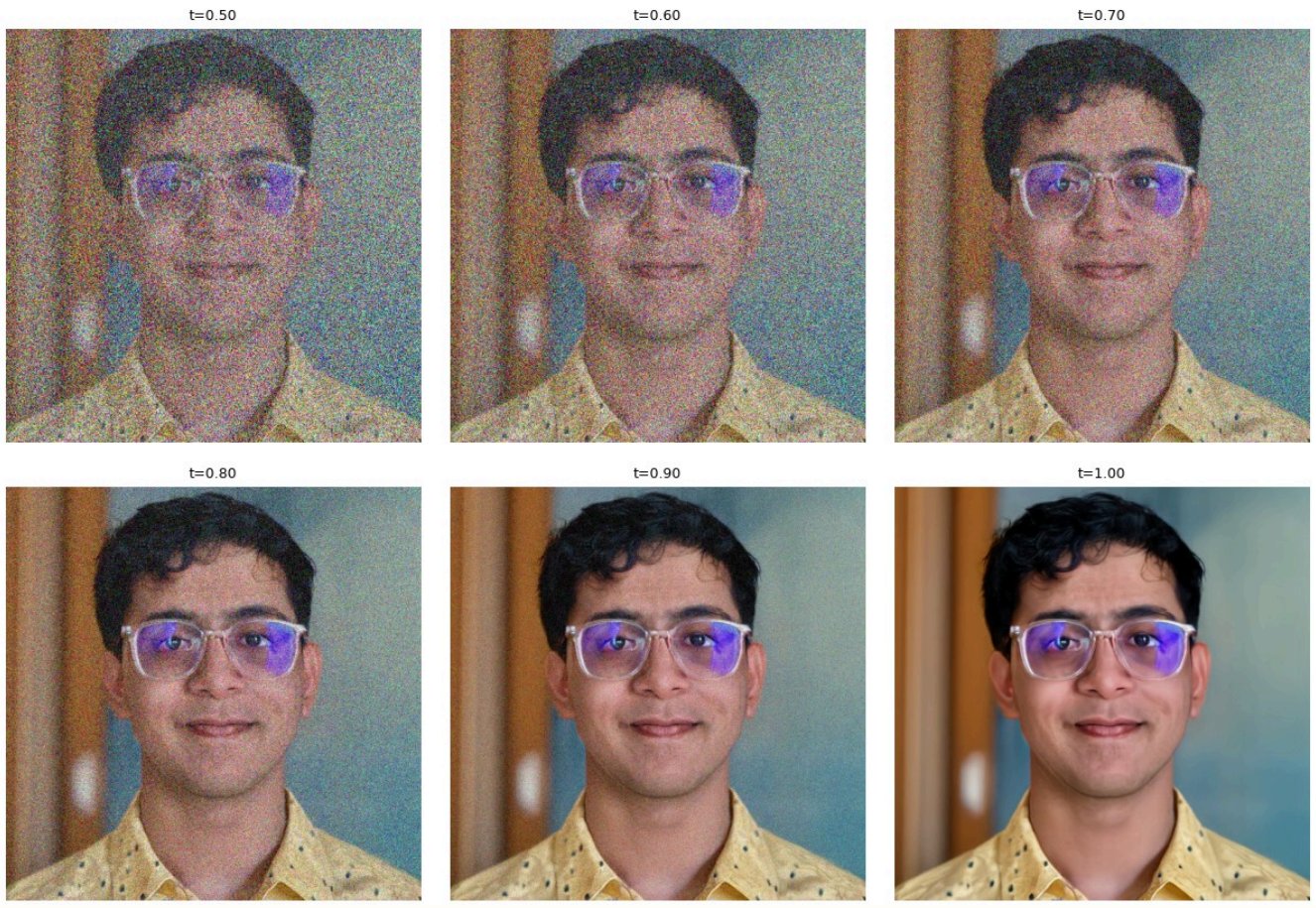


noisy · predicted update · ground-truth update · error at t = 0.75

1.5 Iterative Denoising with Euler Sampling

$$x_{t+h} = x_t + h\, v_\theta(x_t, t, c), \qquad h = 0.02, \text{ starting from } t = 0.50 \text{ (25 updates)}$$

Instead of one big jump, Euler sampling takes small steps of h = 0.02, re-predicting the velocity each time, so going from t = 0.5 to 1 takes 25 updates. Comparing all three on the same input: blur is arguably the worst (artifacts, etc.), one step is somewhat ok, but Euler maintains more of the actual features and looks closer to the original image.

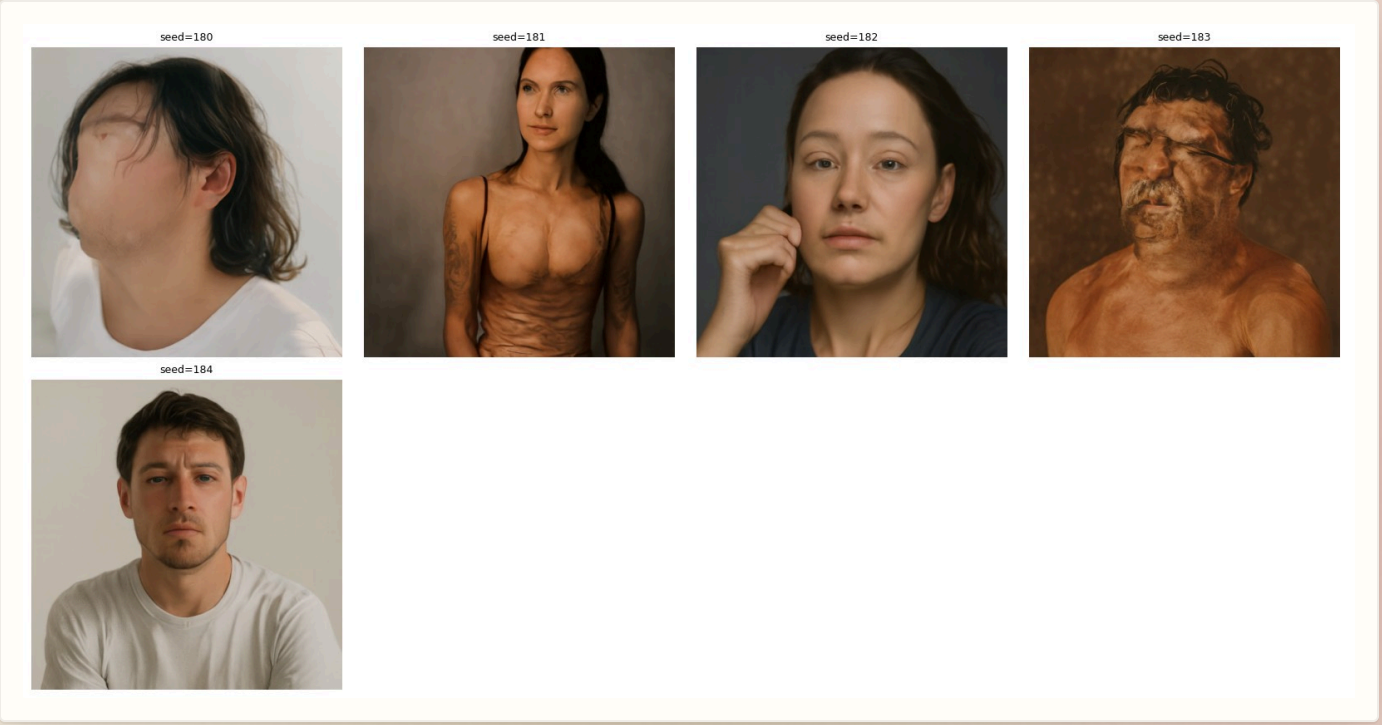


the Euler trajectory, every 5th update: t = 0.50 · 0.60 · 0.70 · 0.80 · 0.90 · 1.00

original · blur · one step · Euler on the same $t = 0.50$ input

1.6 Generating Images from Noise

Starting from pure noise at t = 0 and running all 50 Euler updates with the prompt "a high quality photo" generates images from scratch. These samples look really distorted (they go close to some semblance of the center of the human distribution, which is odd).



five samples, seeds 180 – 184, "a high quality photo"

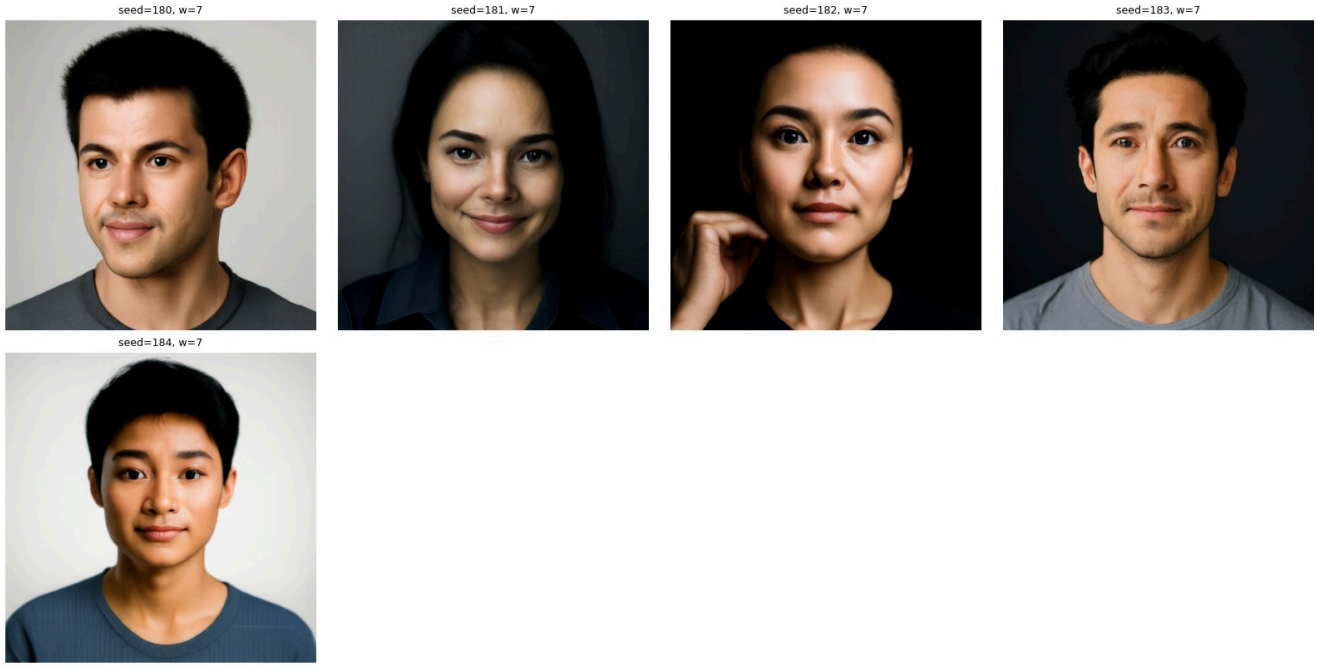
Section 2

Conditional Generation & Guidance

2.1 Classifier-Free Guidance

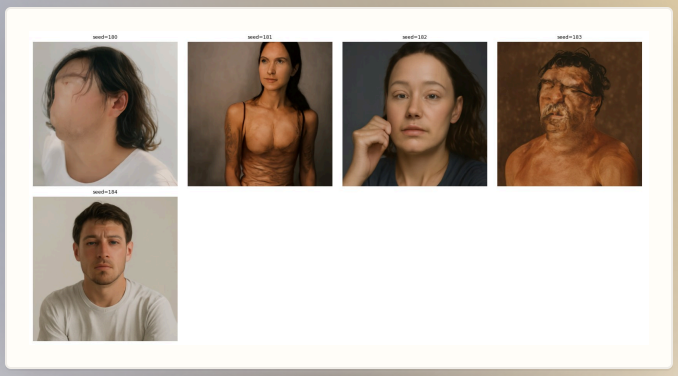
$$v_{cfg} = v_c + w\,(v_c - v_u), \qquad w = 7, \quad v_u \text{ uses the empty prompt}$$

Classifier-free guidance runs the model twice per step, once with the prompt v_c and once with the empty prompt v_u, and pushes further in the direction of the prompt with w = 7. Using the same starting noises as 1.6, CFG makes them look much better: they have fewer artifacts and look human, as opposed to the previous ones.

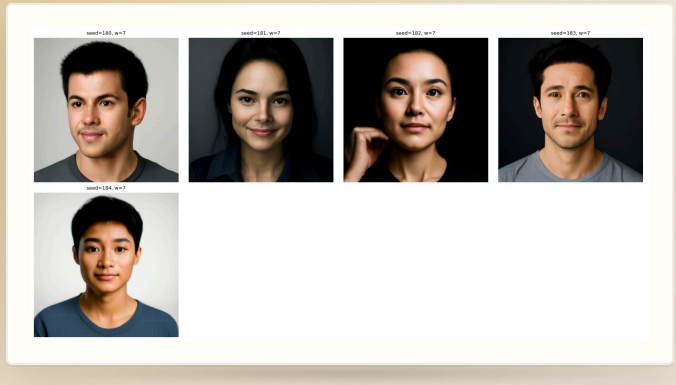


with guidance: seeds 180 – 184, w = 7

Without vs. with guidance



no guidance (1.6)



CFG, $w = 7$

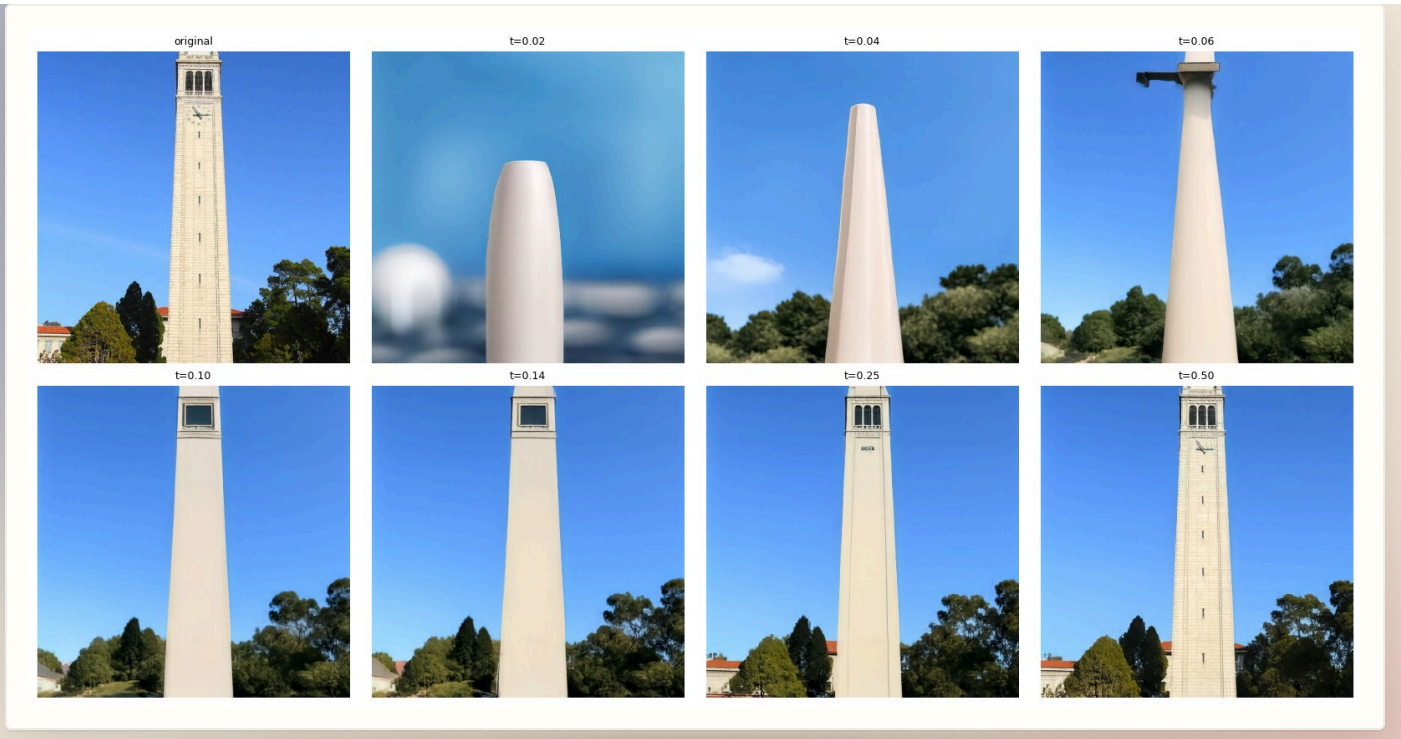
Section 3

Creative Applications

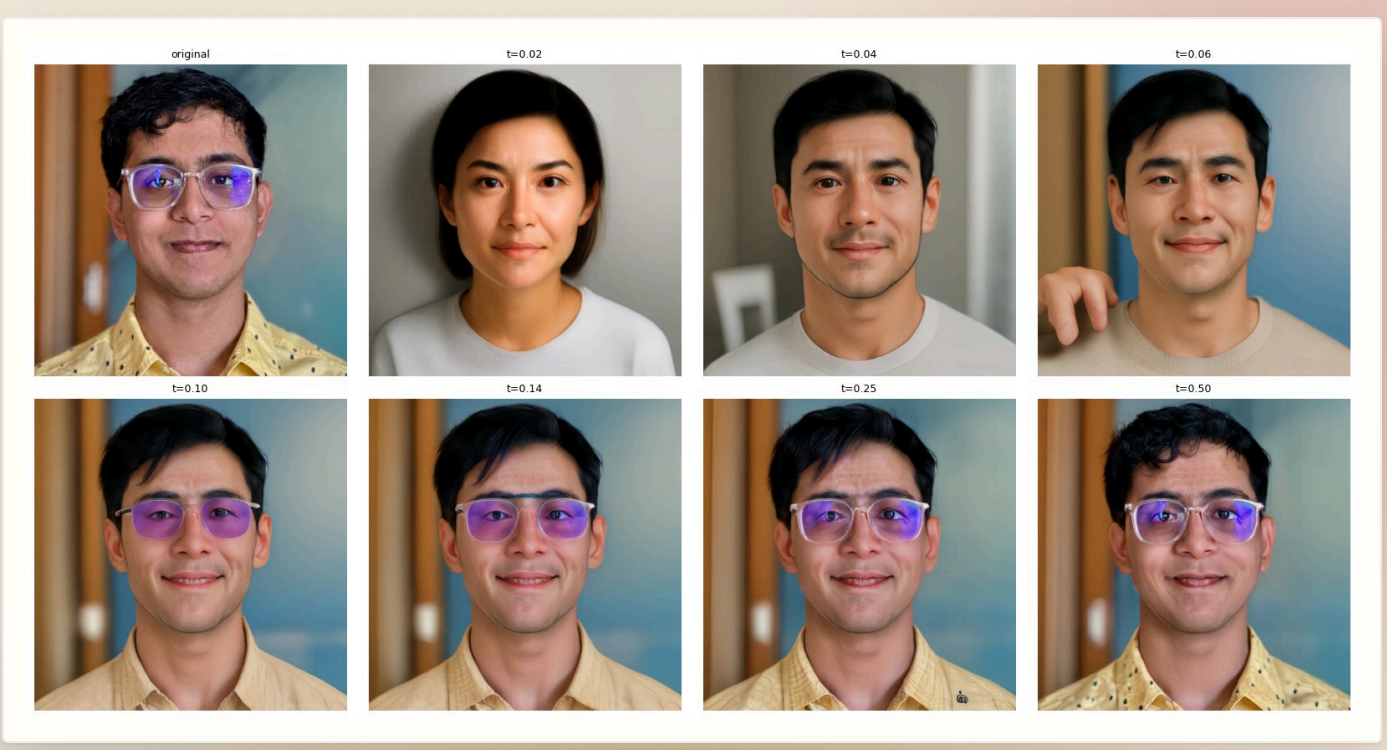
3.1 Image-to-Image Editing

$$x_{t_{start}} = (1 - t_{start})\,\epsilon + t_{start}\,x_1, \qquad t_{start} \in \{0.02,\ 0.04,\ 0.06,\ 0.10,\ 0.14,\ 0.25,$$

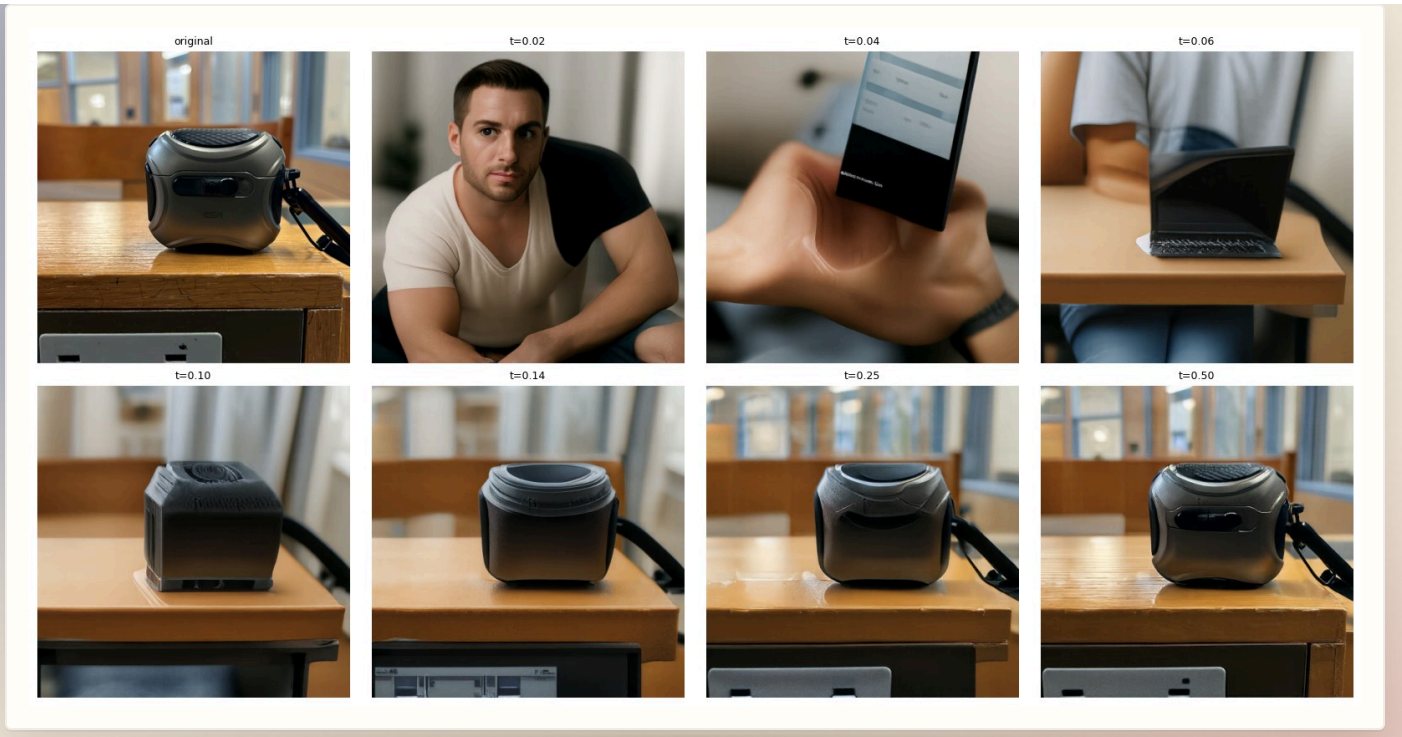
To edit a real image, I noise it to some starting time t_{start} and then run the Euler sampler from there to t = 1. The starting time made the edit more or less pronounced: as we increase the starting time, the edit shows less and less (at an early starting time it's basically a whole different image).



Campanile · original + seven starting times



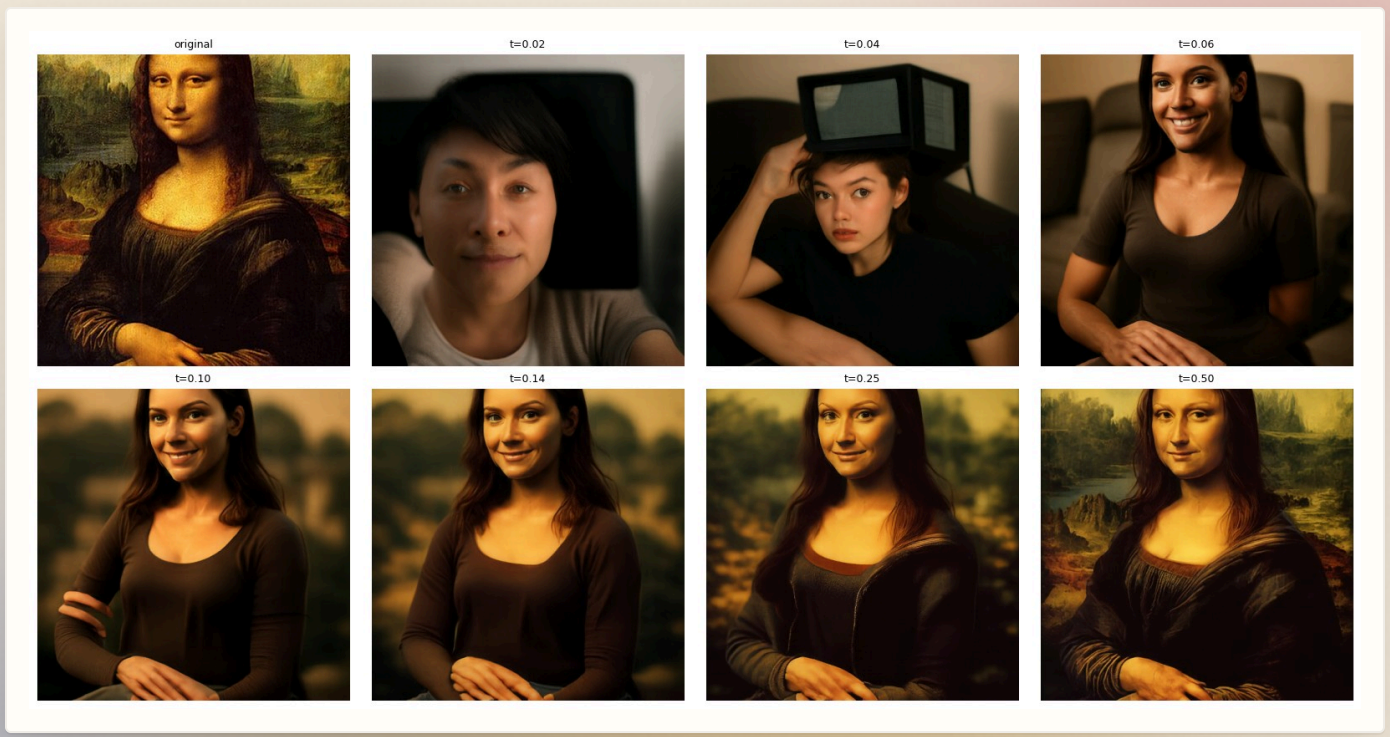
my image 1 · original + seven starting times



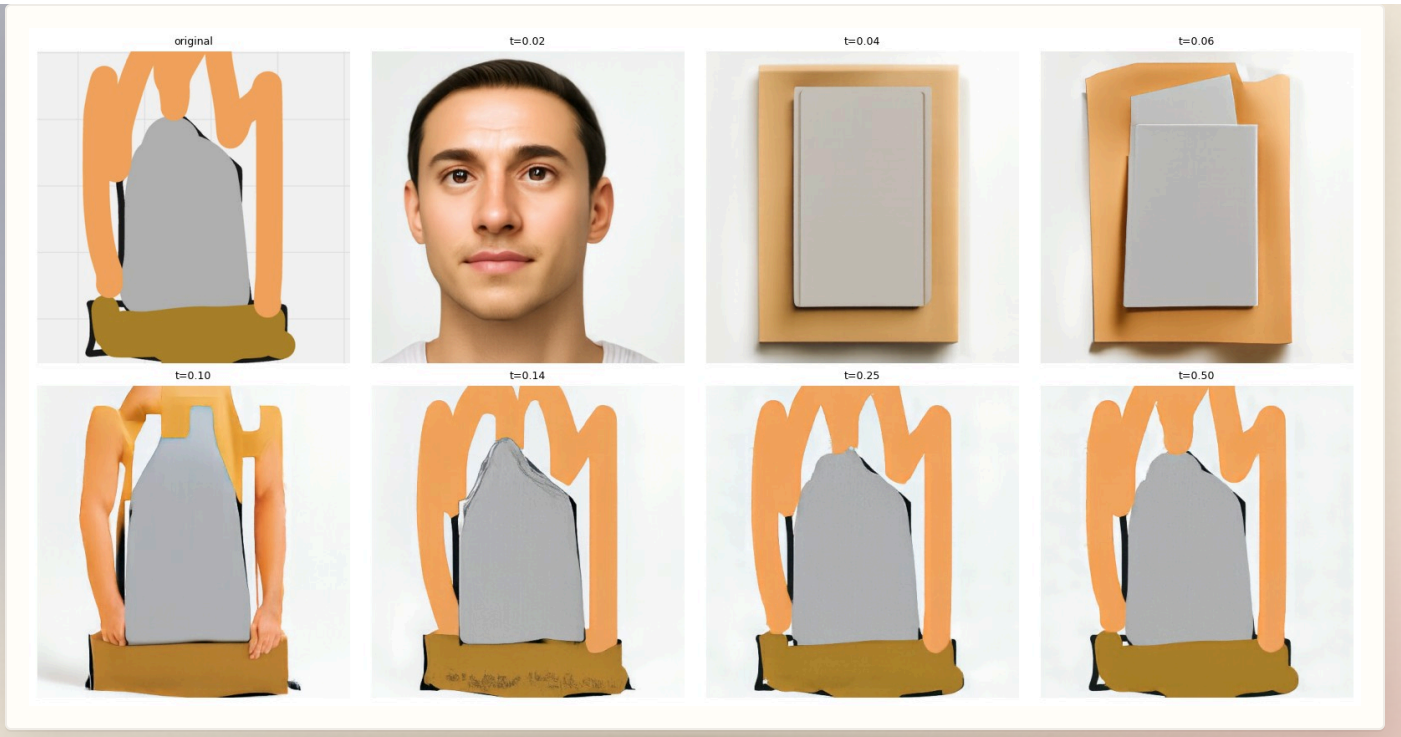
my image 2 · original + seven starting times

Editing hand-drawn & web images

Similar trend for the Mona Lisa and the drawings. Here there is some time t that is more optimal for the edit, and it's a tradeoff.



web image · original + seven starting times

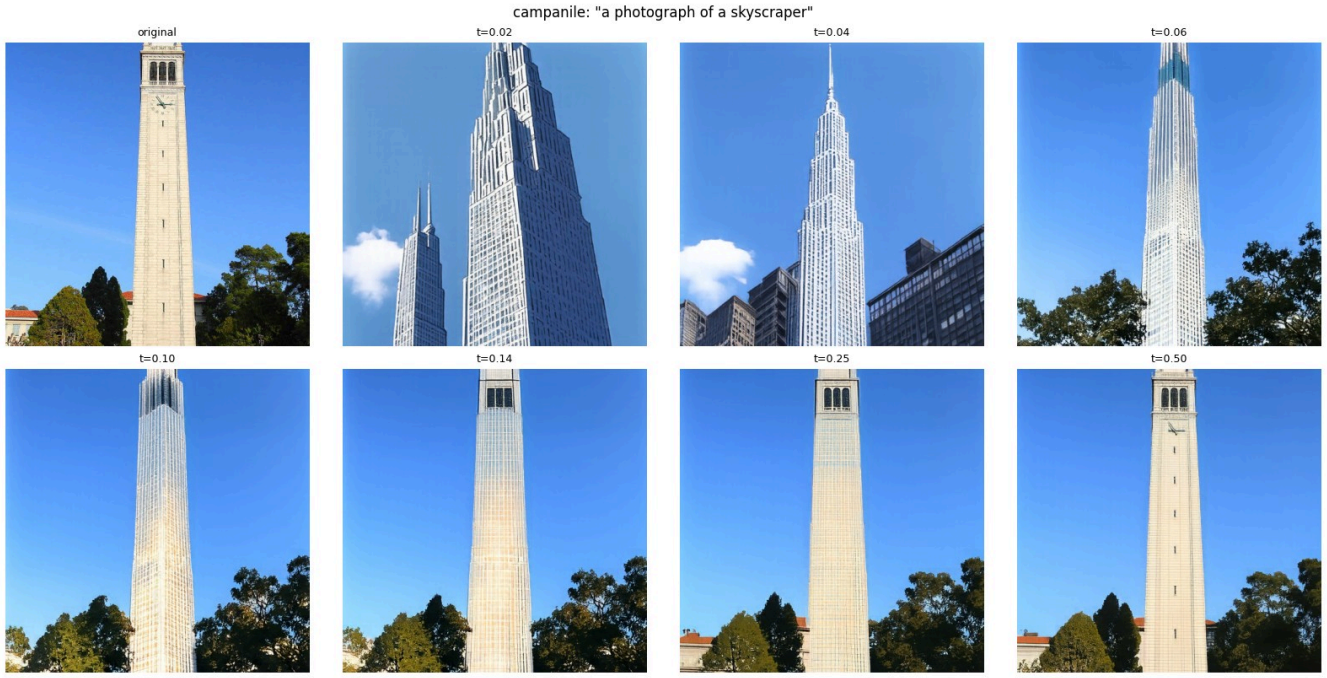


drawing 1 · original + seven starting times

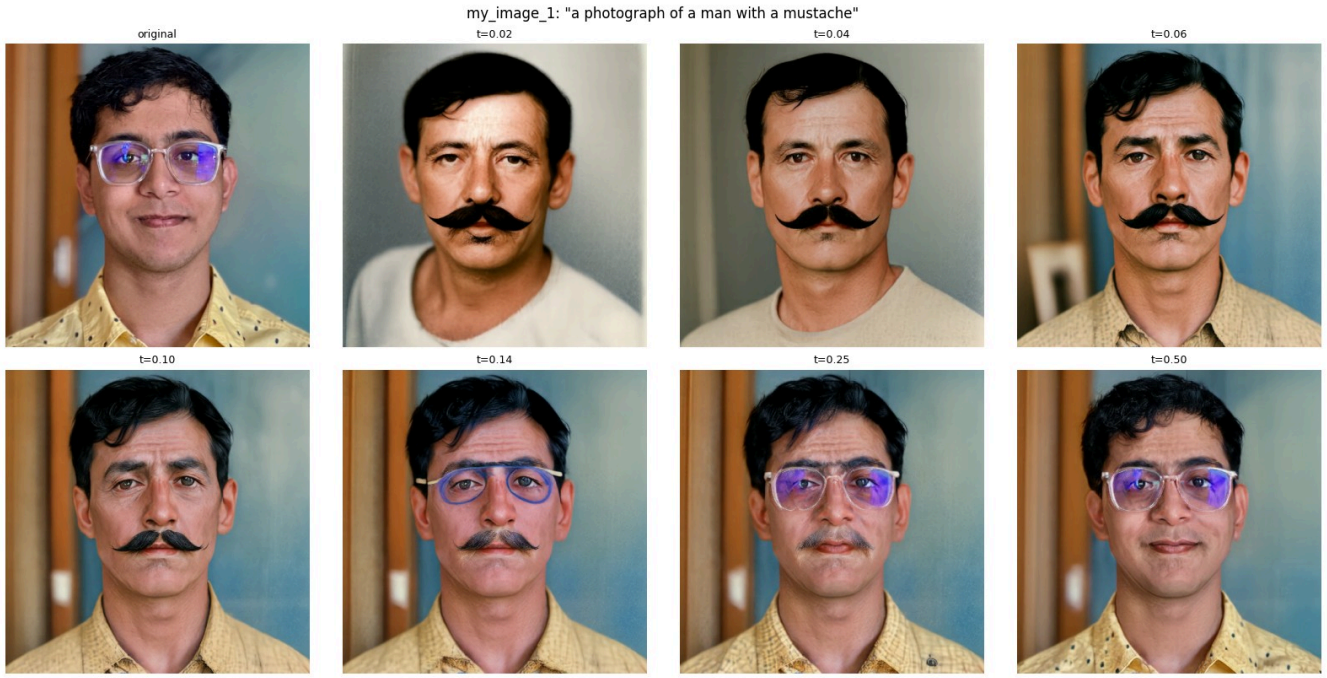
drawing 2 · original + seven starting times

3.2 Text-Guided Image Editing

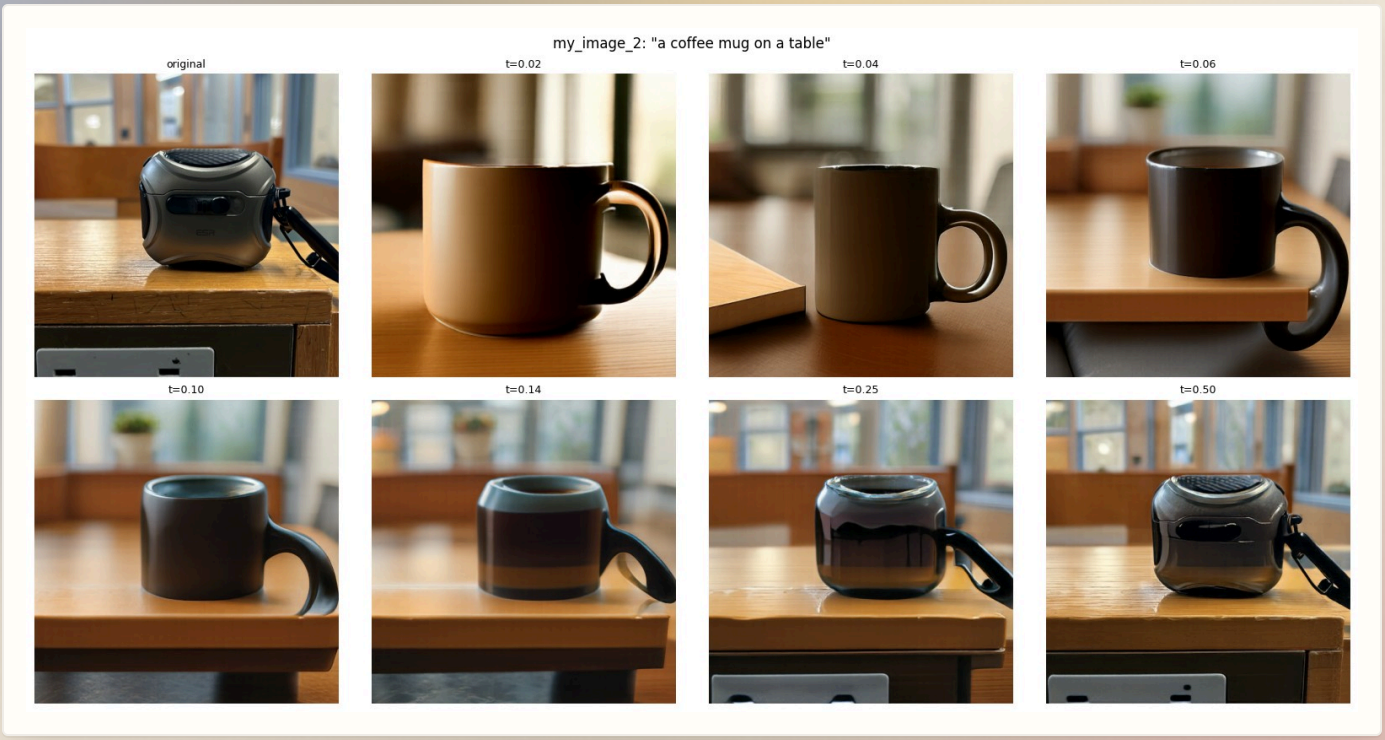
Same process as 3.1, but instead of "a high quality photo" I use an edit prompt, so the model pulls the image toward that text. The edit did ok (it seems like around t = 0.10 is where we start to see the beginning of adherence to the actual image, and past that it gets closer and closer).



Campanile → "a photograph of a skyscraper"



my image 1 → "a photograph of a man with a mustache"



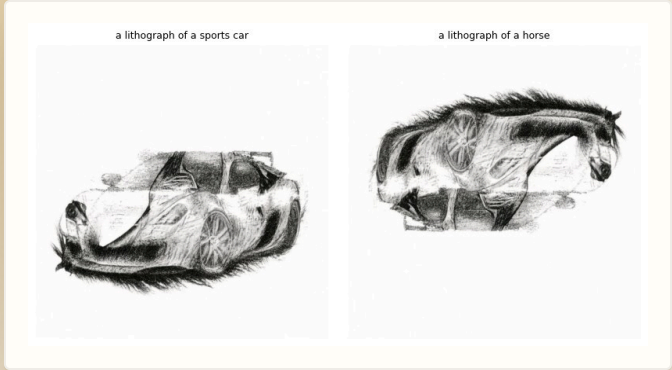
my image 2 → "a coffee mug on a table"

3.3 Visual Anagrams

$$v = \tfrac{1}{2}\Big[v_{cfg}(x, t, c_a) + F\big(v_{cfg}(F(x), t, c_b)\big)\Big], \qquad F = \text{rotate } 180^\circ$$

At every step I take the CFG velocity for the upright image with one prompt, plus the velocity for the image rotated 180° with the other prompt (rotated back), and average the two. That way the image has to make sense both ways. The anagrams are cool: the old man and woman work well, while the car and horse is okay, but the horse is a bit too embedded.

upright: "an oil painting of a grumpy old man"
rotated: "an oil painting of a happy woman"



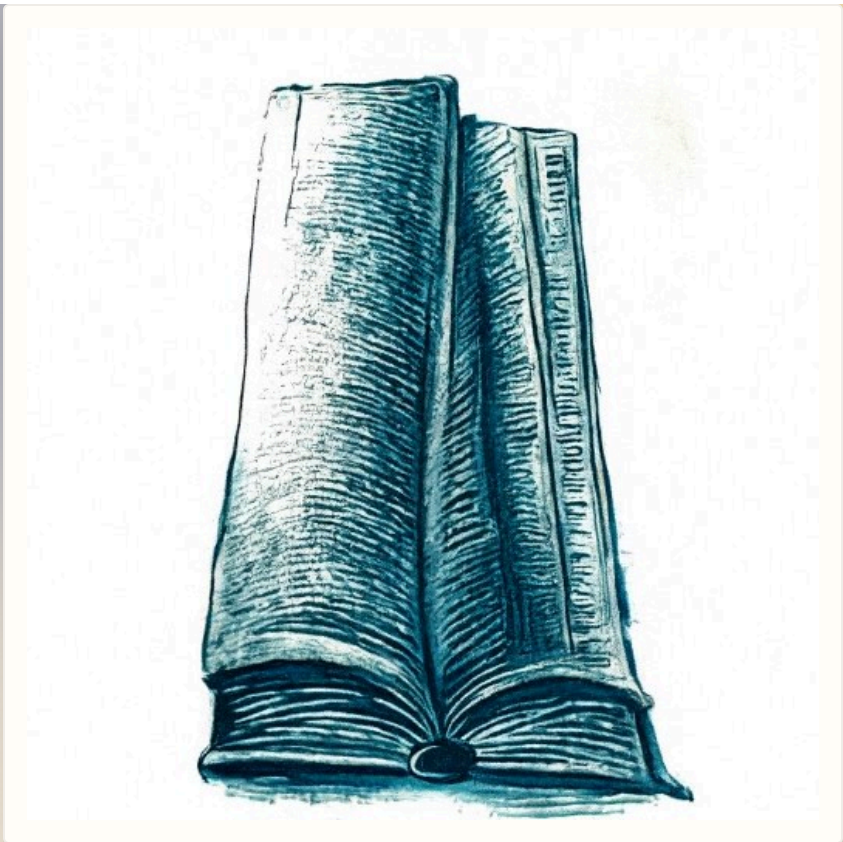
upright: "a lithograph of a sports car"
rotated: "a lithograph of a horse"

3.4 Hybrid Images

$$v = \text{lowpass}(v_a) + v_b - \text{lowpass}(v_b), \qquad \text{Gaussian } \sigma = 16,\ \text{kernel } 265$$

At every step I combine the low frequencies of one prompt's velocity with the high frequencies of the other's (Gaussian σ = 16). Up close you see the high-frequency prompt, and from far away (or tiny, like the 64 px version) the low-frequency one takes over. These are also cool. Both lithographs seem to work nicely, though the book is a little lengthy and the bread is an interesting shape.

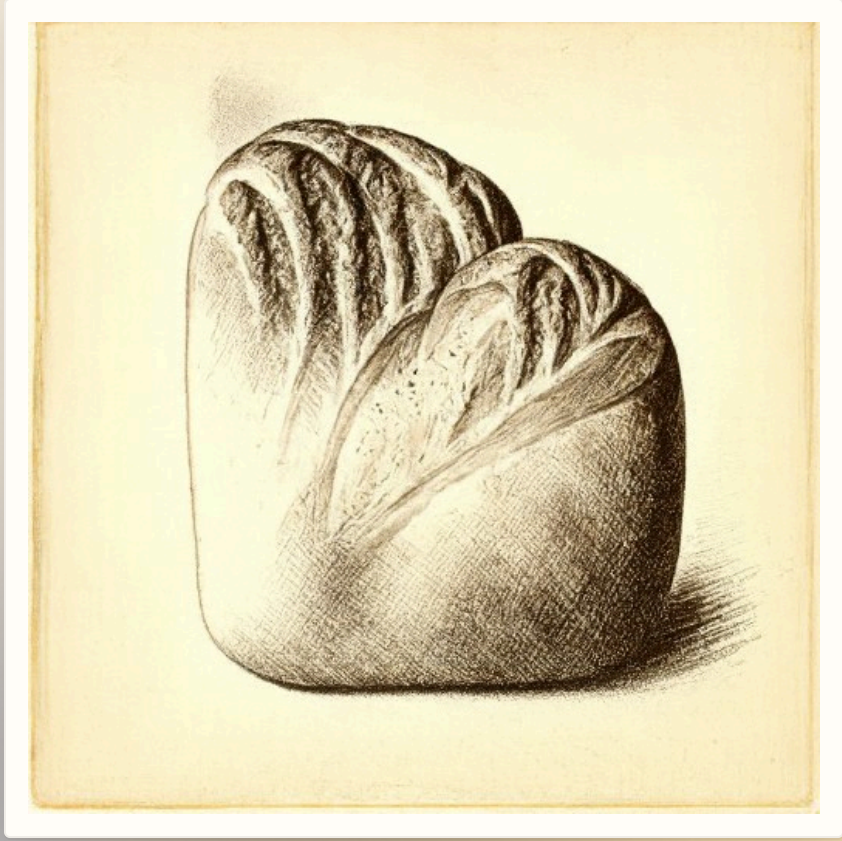
Skyscraper / Book

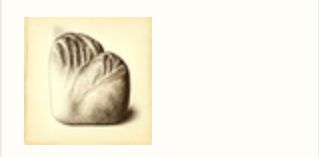


close: "a lithograph of a book"

distant:
"a lithograph of a skyscraper"

Cat / Bread





close: "a lithograph of a bread"

distant: "a lithograph of a cat"

Part B

Flow Matching from Scratch

Part B is building a UNet from scratch.

data **MNIST, pixels in [0, 1]** GPU **NVIDIA L4** seed **180** velocity target $x_1 - x_0$ guidance $\gamma = 5$

Part B · Section 1

Training a Single-Step Denoising UNet

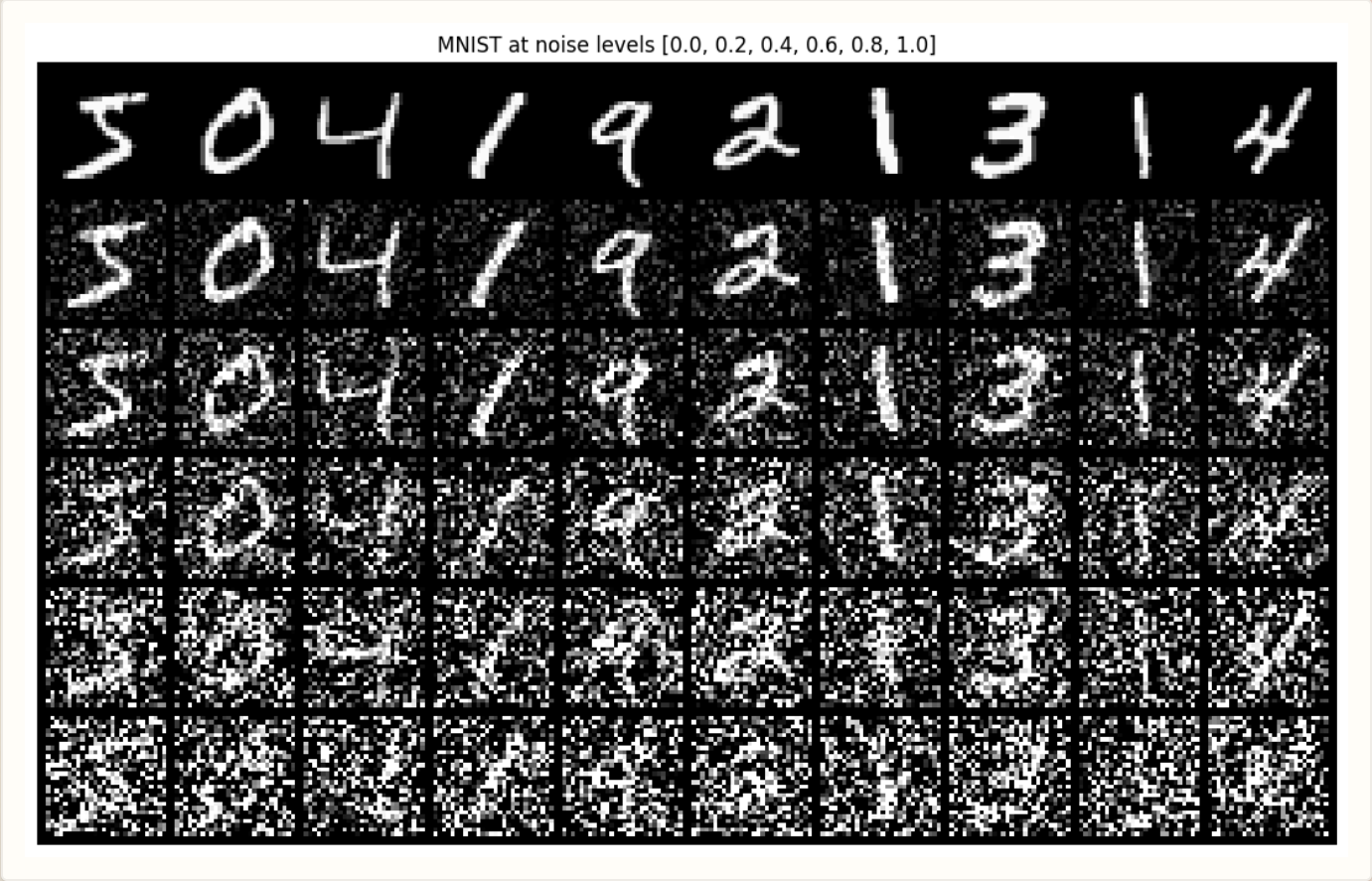
1.1 Implementing the UNet

The UNet is built from a few simple ops: Conv (3×3 conv, BatchNorm, GELU), DownConv (the same but stride 2, so it halves the size), UpConv (a 4×4 transposed conv that doubles it), Flatten (a 7×7 average pool down to 1×1) and Unflatten (a 7×7 transposed conv back up to 7×7). The encoder goes 28 → 14 → 7 → 1, the decoder mirrors it, and at each level the decoder concatenates the matching encoder features as a skip connection. A last 3×3 conv maps the D hidden channels back to one channel.

1.2 Using the UNet to Train a Denoiser

$$z = x + \sigma\,\epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \qquad L = \mathbb{E}\,\|D_\theta(z) - x\|^2$$

To make training pairs I take a clean digit x and add Gaussian noise, $z = x + \sigma\epsilon$, and the denoiser learns to map z back to x with an L2 loss. The grid shows the same ten digits as σ goes from 0 to 1.



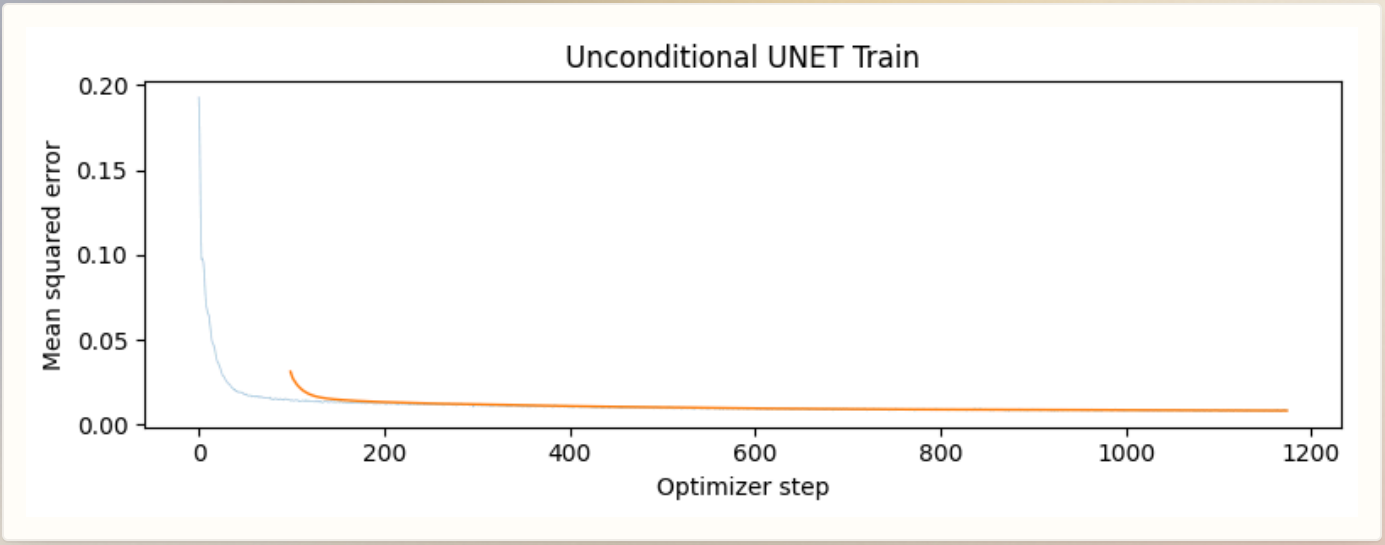
rows: σ = 0.0 · 0.2 · 0.4 · 0.6 · 0.8 · 1.0

1.2.1 · Training

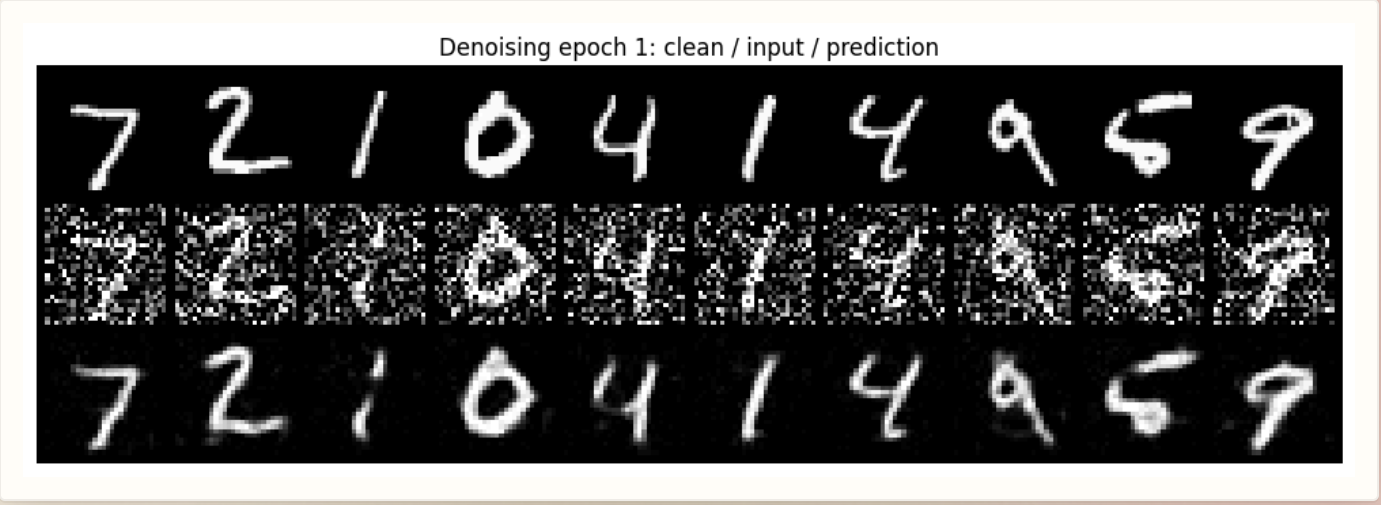
σ 0.5 | D 128 | batch 256 | Adam lr 1e-4 | epochs 5

During epoch 1 the images still resemble the shapes, but they tend to be a little blurry, missing features, or otherwise imperfect. After epoch 5 we see less of this (although if

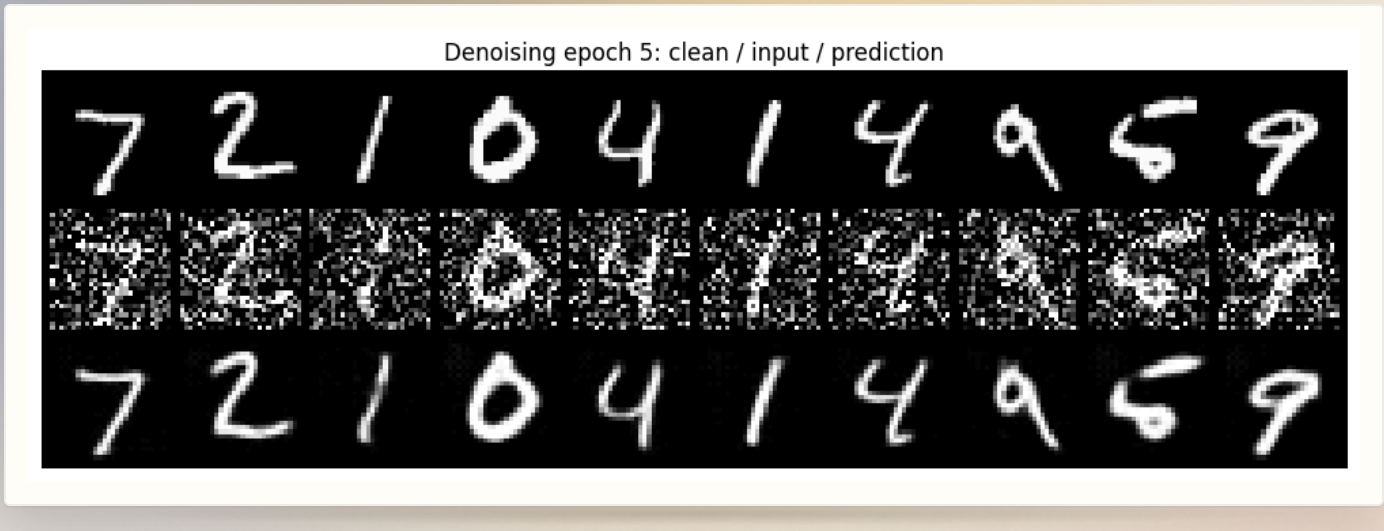
you look at the one in the middle, it still has some fade to it).



training loss



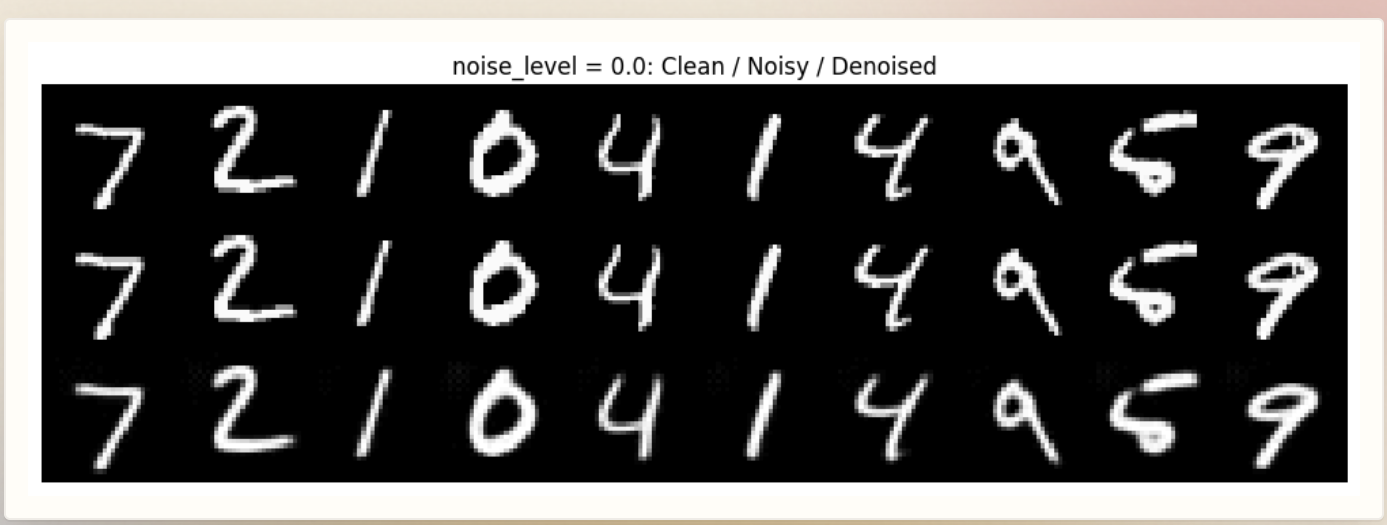
after epoch 1 · clean / noisy / denoised



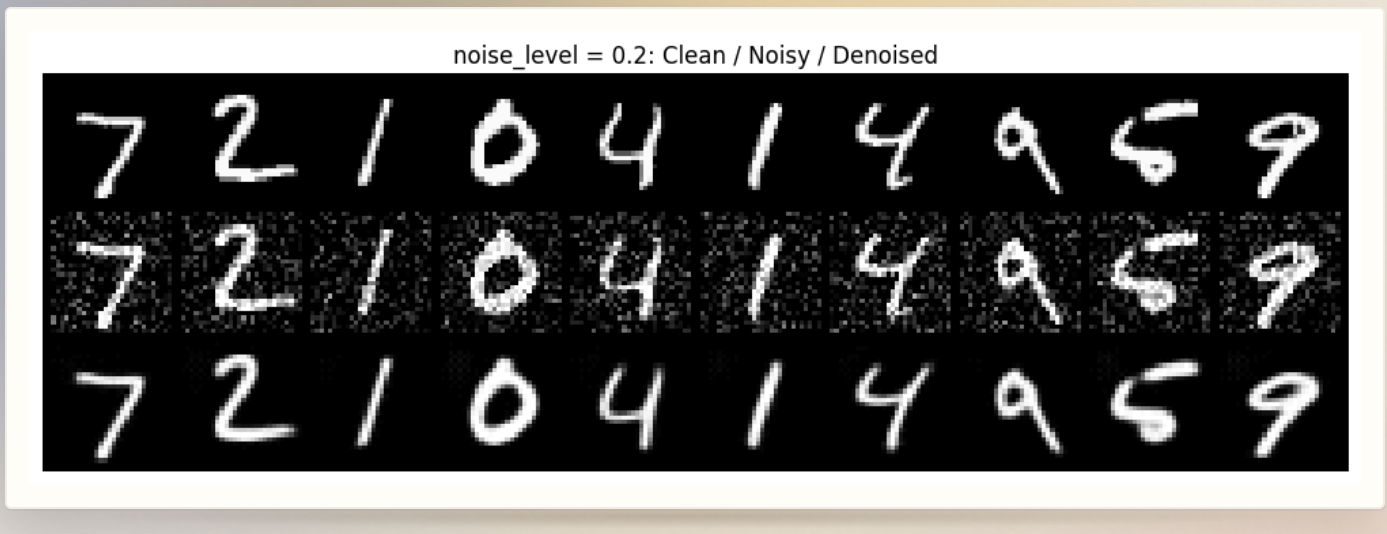
after epoch 5 · clean / noisy / denoised

1.2.2 · Out-of-Distribution Testing

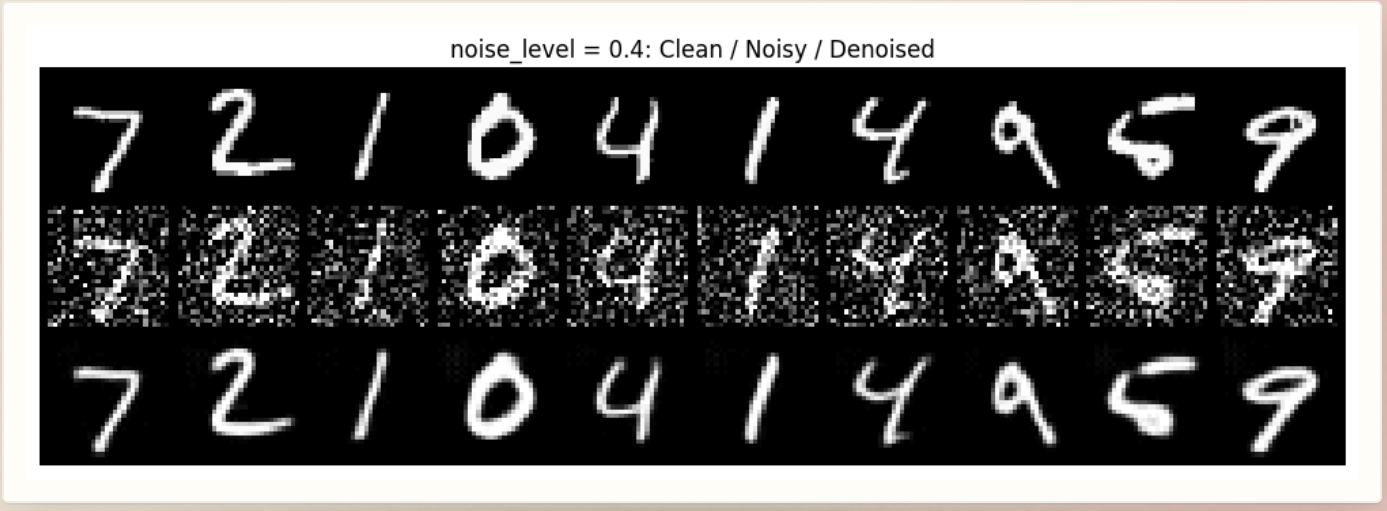
σ = 0.6 is the last level where it looks good. After that point (0.8) the images still kind of look like the original, but we see way more artifacts and the objects become blobby. I think it fails there as opposed to an earlier step because it was trained at σ = 0.5, so noise on top of that probably isn't accounted for, and the model "learns" that it might've been part of the original image, thus leading to the artifacts we see.



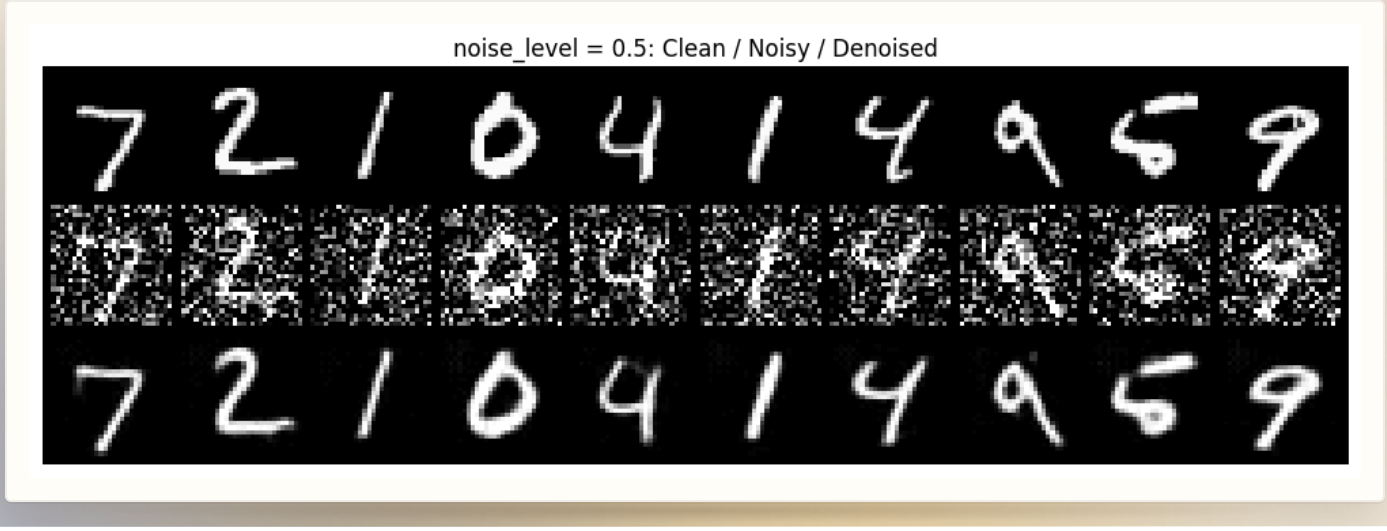
σ = 0.0 · clean / noisy / denoised



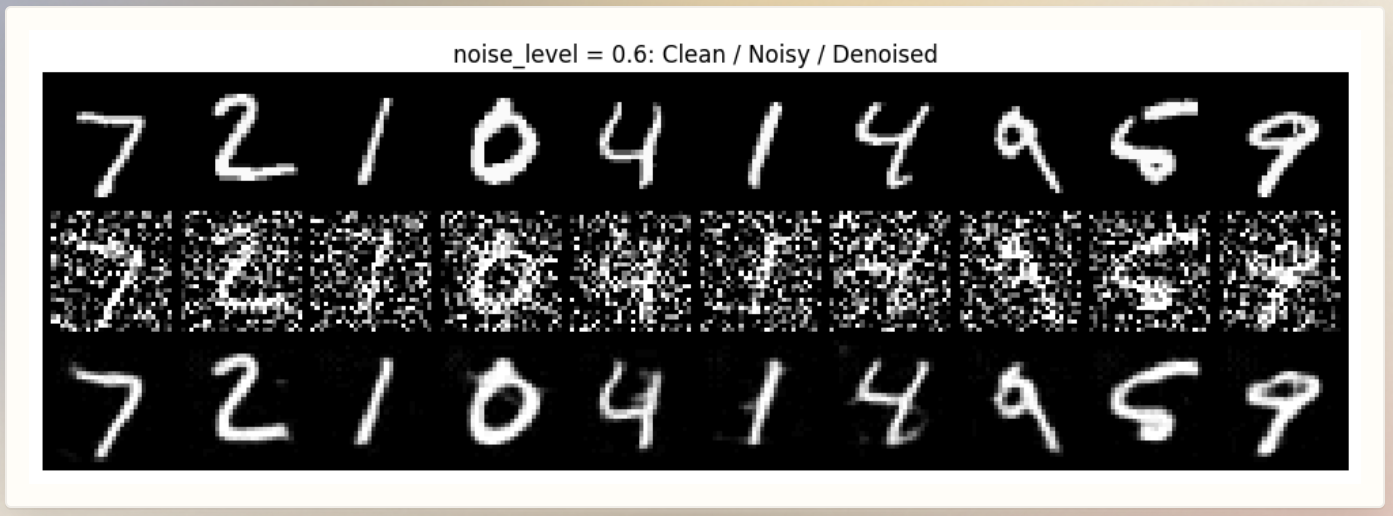
$\sigma = 0.2$ · clean / noisy / denoised



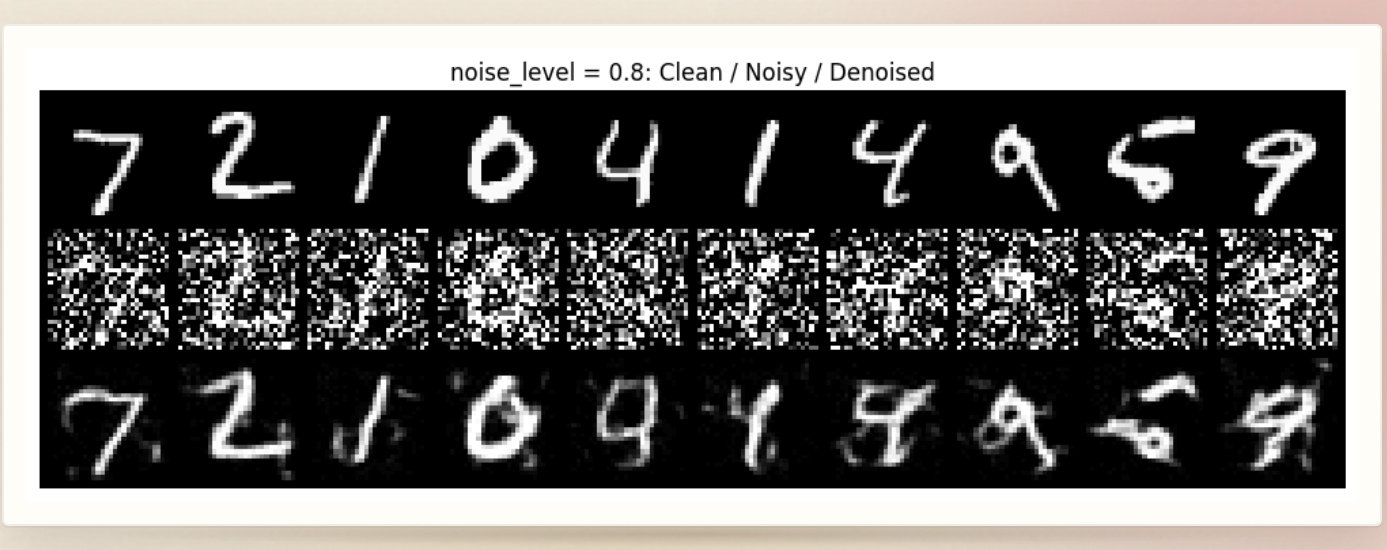
$\sigma = 0.4$ · clean / noisy / denoised



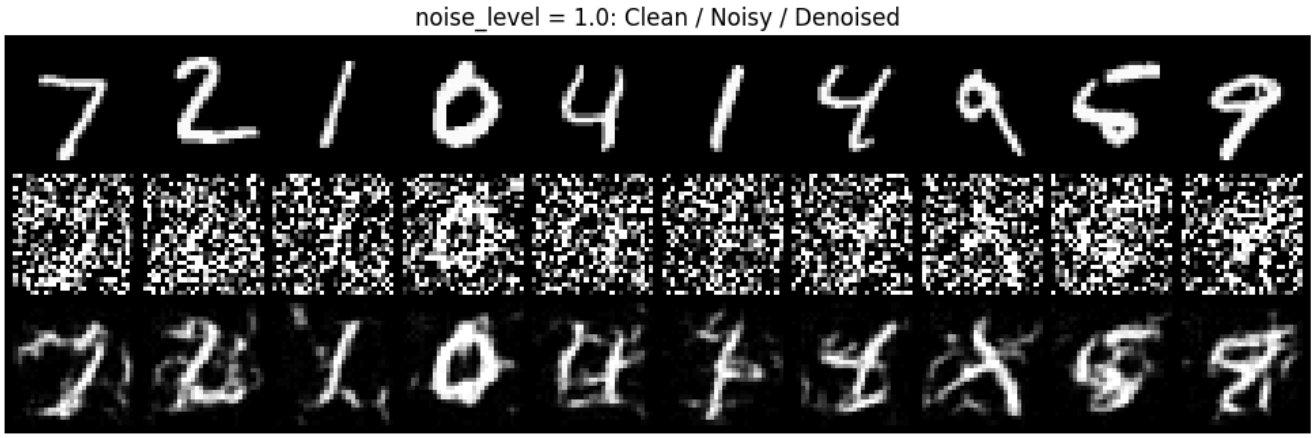
σ = 0.5 · clean / noisy / denoised



σ = 0.6 · clean / noisy / denoised



σ = 0.8 · clean / noisy / denoised

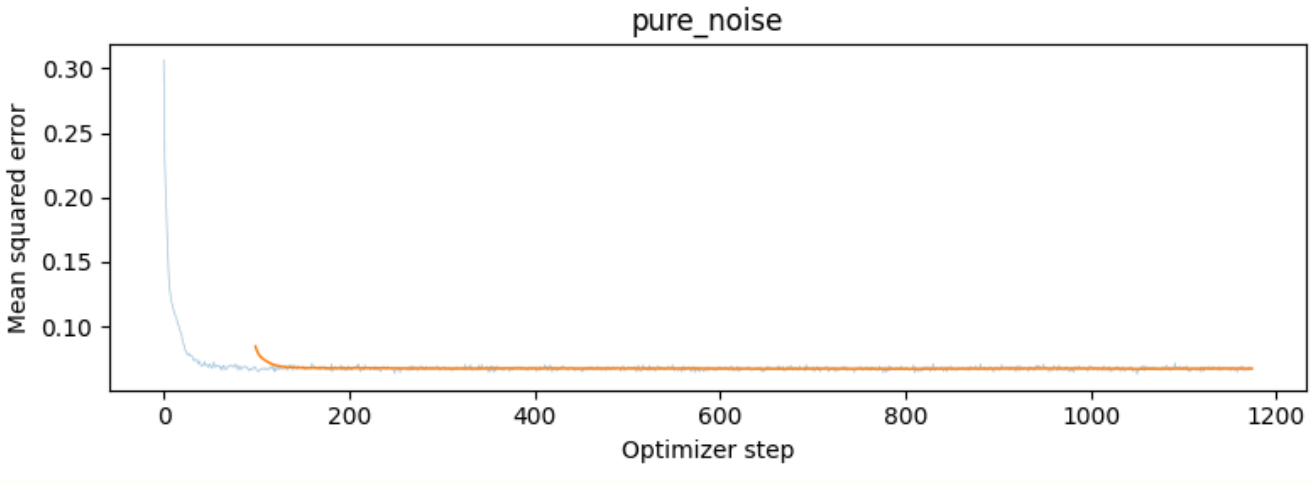


σ = 1.0 · clean / noisy / denoised

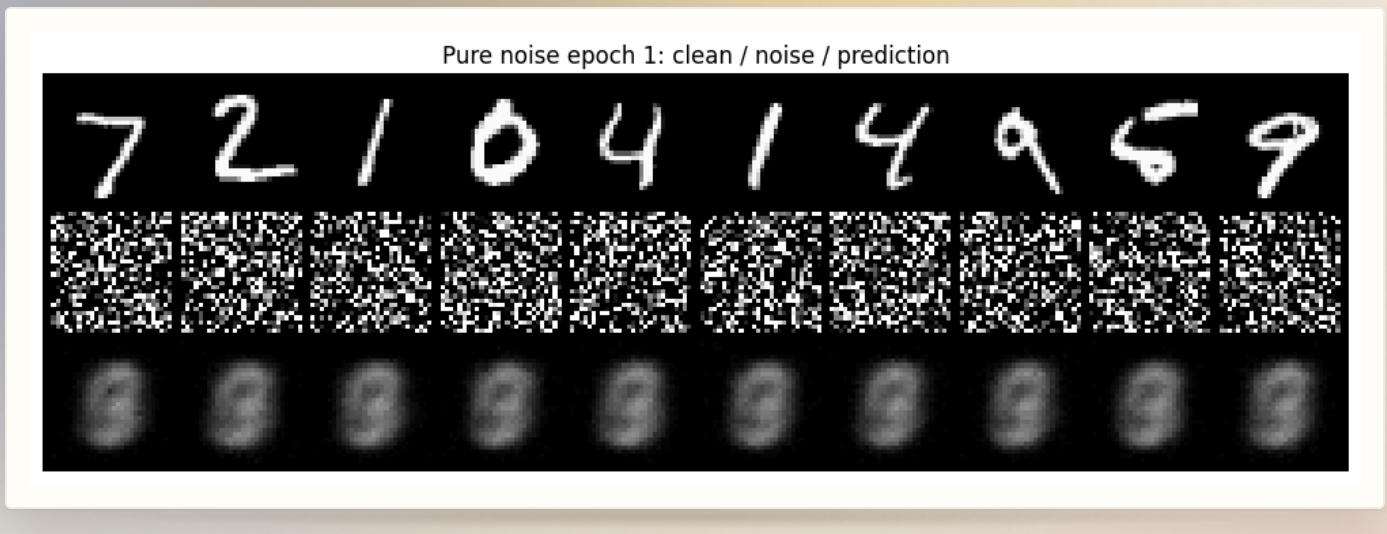
1.2.3 · Denoising Pure Noise

$$z = \epsilon \sim \mathcal{N}(0, I), \qquad L = \mathbb{E}\,\|D_\theta(\epsilon) - x\|^2$$

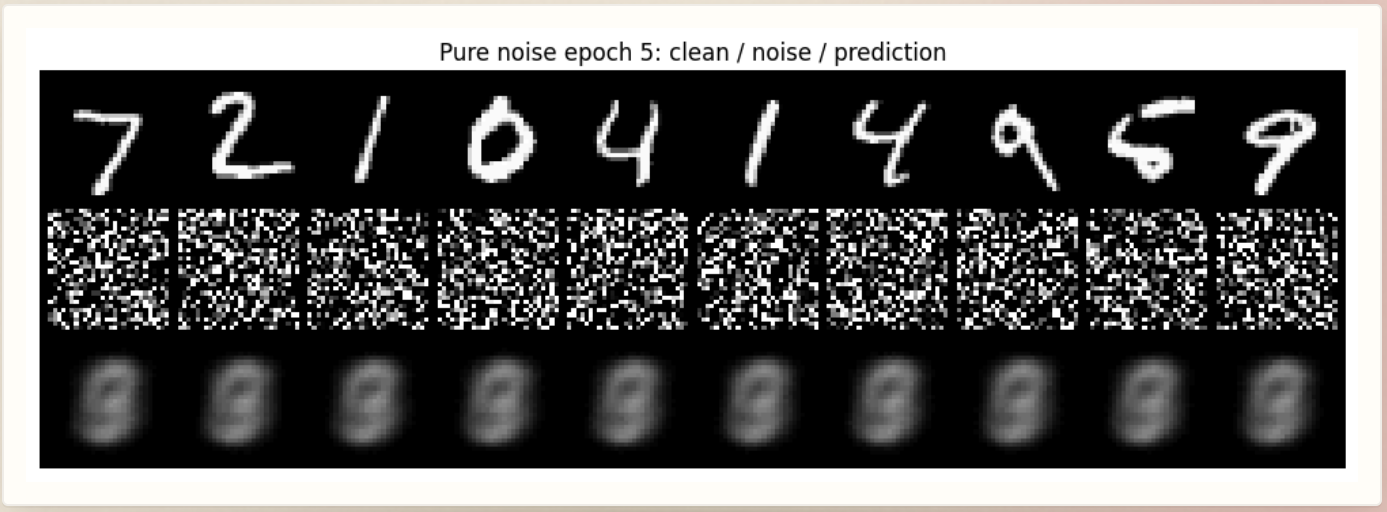
It looks like the average of all the digits. Since the input is all noise, across all samples the image provides no general starting point, so seeing noise, it doesn't really know where to go. Thus, to minimize MSE as per usual, it just chooses the average, which is a feature.



training loss on pure noise



after epoch 1 · clean / noise / prediction



after epoch 5 · clean / noise / prediction

Part B · Section 2

Flow Matching

2.1 Implementing a Time-Conditioned UNet

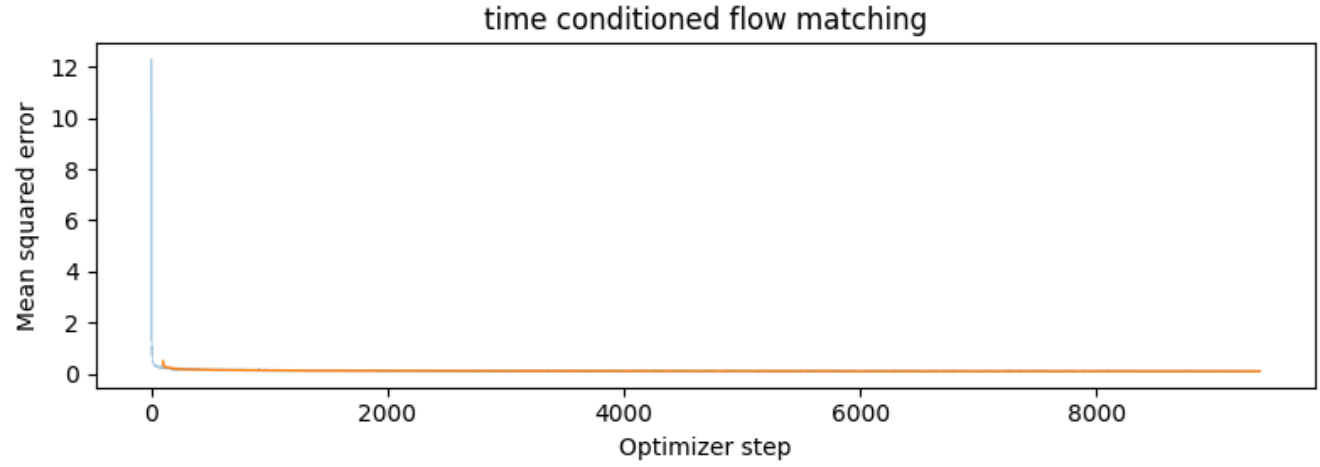
$$x_t = (1-t)\,x_0 + t\,x_1, \qquad x_0 \sim \mathcal{N}(0, I), \qquad L = \mathbb{E}\left\|u_\theta(x_t, t) - (x_1 - x_0)\right\|^2$$

Instead of denoising in one step, the UNet now takes a point x_t on the straight line between noise x_0 and a digit x_1 and predicts the velocity $x_1 - x_0$ that pushes it toward the digit. The time t goes through two FCBlocks (Linear, GELU, Linear), and their outputs scale the unflattened features and the output of the first up block.

2.2 Training the Time-Conditioned UNet

D **64** batch **64** Adam lr **1e-2** ExponentialLR $\gamma = 0.1^{1/10}$ epochs **10**

Each step picks a random $t \in [0, 1]$ per image, builds x_t from fresh noise, and trains on the MSE between the predicted and true velocity. The learning rate decays exponentially so it ends 10× lower after the 10 epochs.

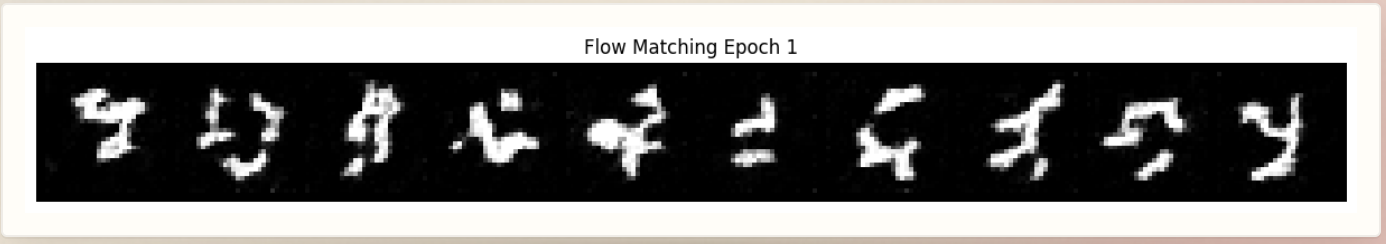


time-conditioned flow matching loss

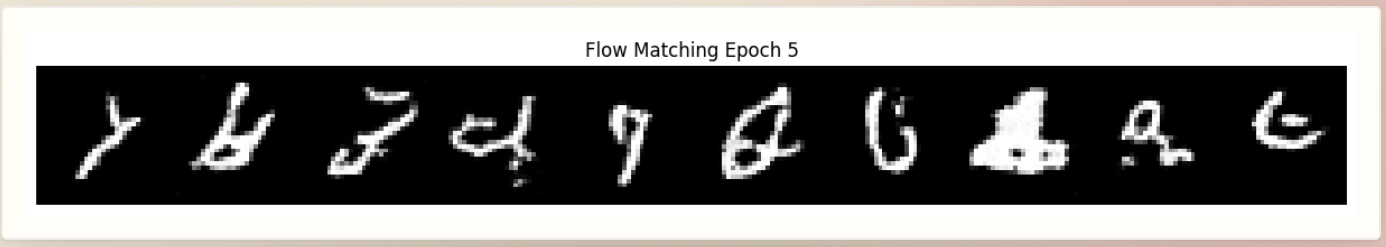
2.3 Sampling from the Time-Conditioned UNet

$$x_{t+\Delta t} = x_t + \Delta t\, u_\theta(x_t, t), \qquad \Delta t = 1/50, \;\; t : 0 \to 1$$

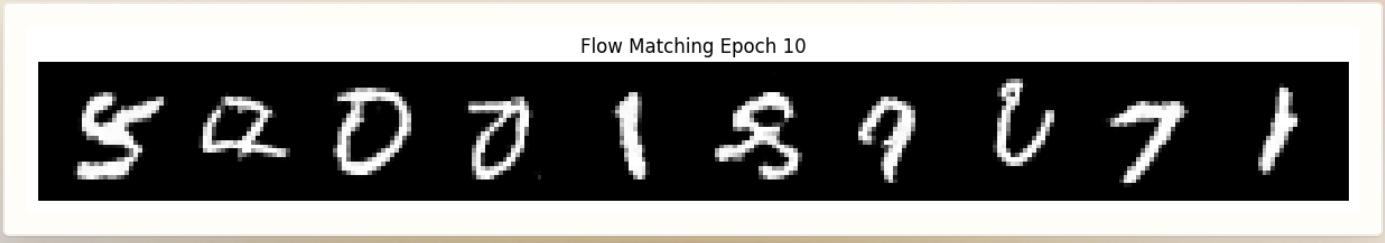
After epoch 1 they don't look that good, at epoch 5 they become a bit more recognizable, and after epoch 10 I can tell what each digit is, but it still looks like it was written illegibly (i.e. the handwriting isn't good). This is even more so on the digits with more complex twists and turns, such as the 8.



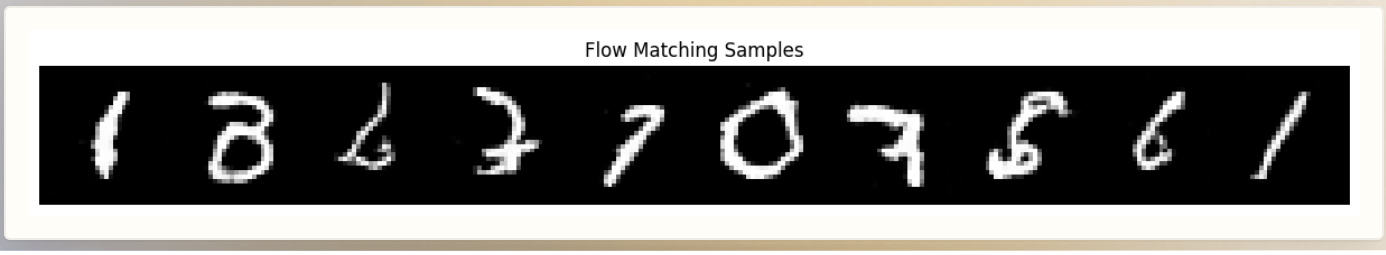
after epoch 1



after epoch 5



after epoch 10



final samples

2.4 Implementing a Class-Conditioned UNet

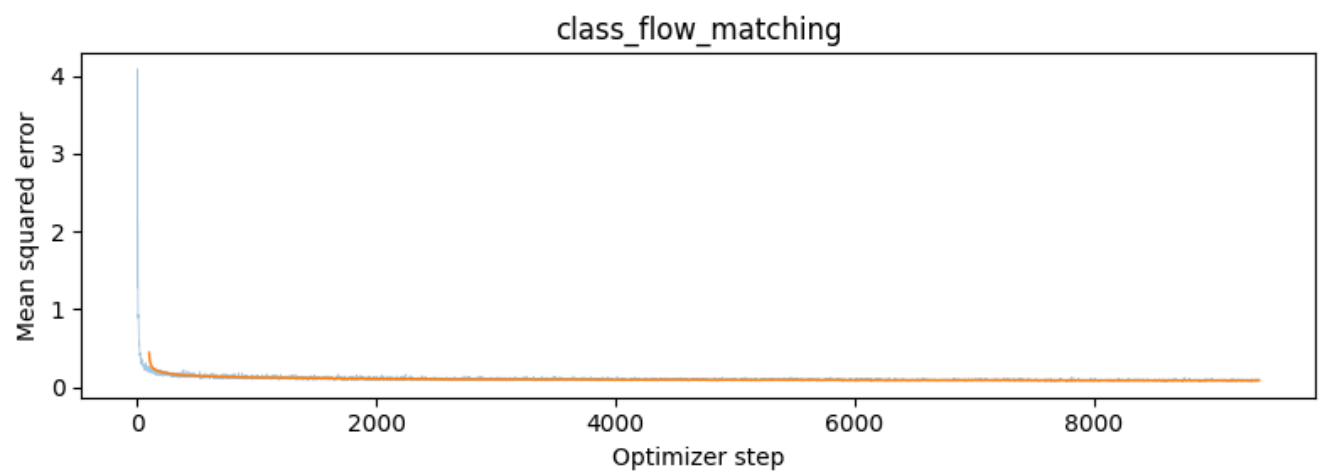
$$\text{unflat} \leftarrow c_1 \odot \text{unflat} + t_1, \qquad \text{up}_1 \leftarrow c_2 \odot \text{up}_1 + t_2, \qquad c \to \mathbf{0} \text{ with } p_{uncond} =$$

On top of the time conditioning, the digit label goes in as a one-hot vector c through two more FCBlocks. The class embedding scales the features and the time embedding shifts them. 10% of the time the class vector is zeroed out, so the same model also learns the unconditional velocity, which is what classifier-free guidance needs at sampling time.

2.5 Training the Class-Conditioned UNet

D **64** batch **64** Adam lr **1e-2** ExponentialLR $\gamma = 0.1^{1/10}$ p_{uncond} **0.1** epochs **10**

Same training loop as 2.2, just with the labels passed in and class dropout at $p_{uncond} = 0.1$.

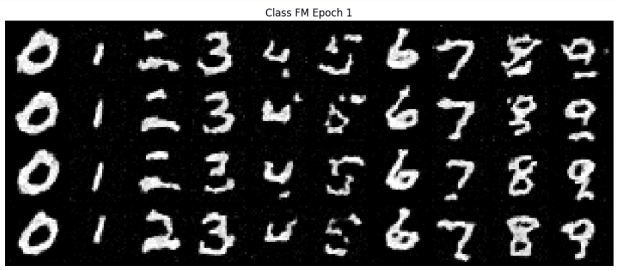


class-conditioned loss, with scheduler

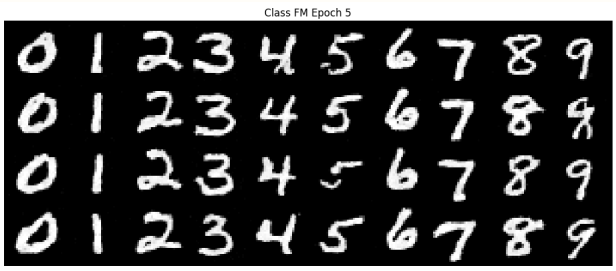
2.6 Sampling from the Class-Conditioned UNet

$$u = u_u + \gamma\,(u_c - u_u), \qquad \gamma = 5, \quad 300 \text{ Euler steps}$$

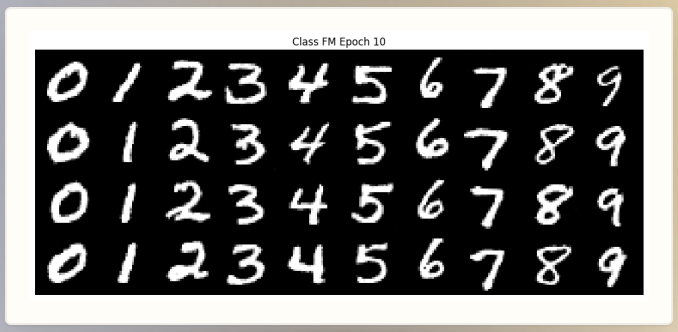
With class conditioning, epoch 1 already looks pretty good with some noise artifacts, epoch 5 is almost perfect (again, some artifacts on the 9), and epoch 10 is practically perfect. This is so much better than 2.3.



after epoch 1 · four of each digit



after epoch 5 · four of each digit

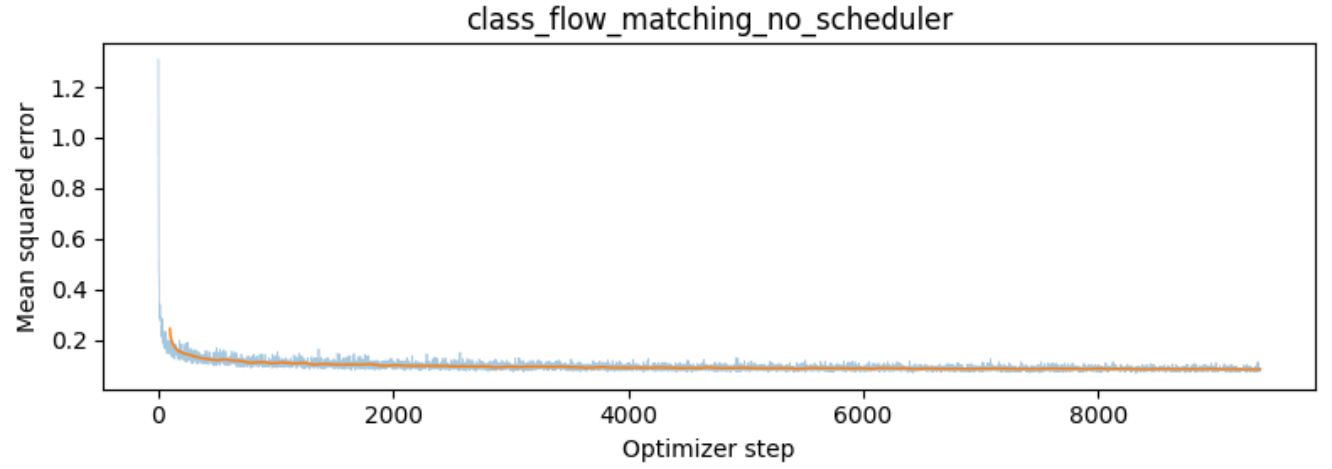


after epoch 10 · four of each digit

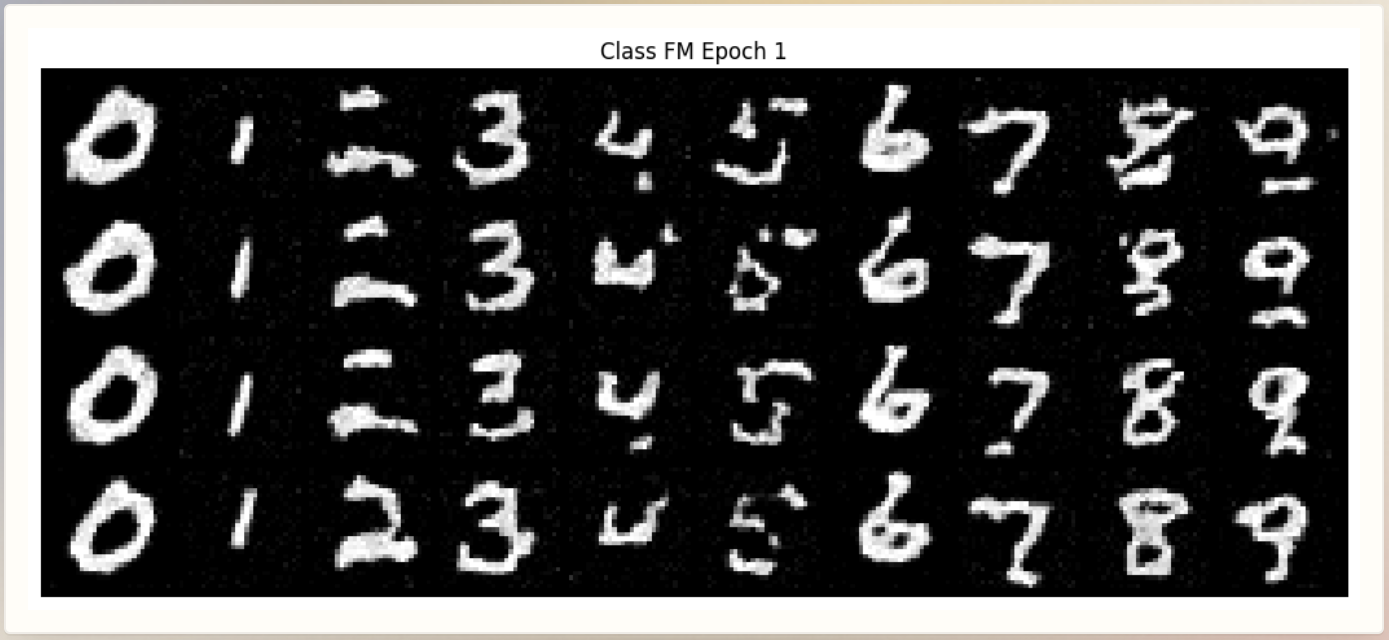
Getting rid of the scheduler

constant Adam lr **1e-3** epochs **10**

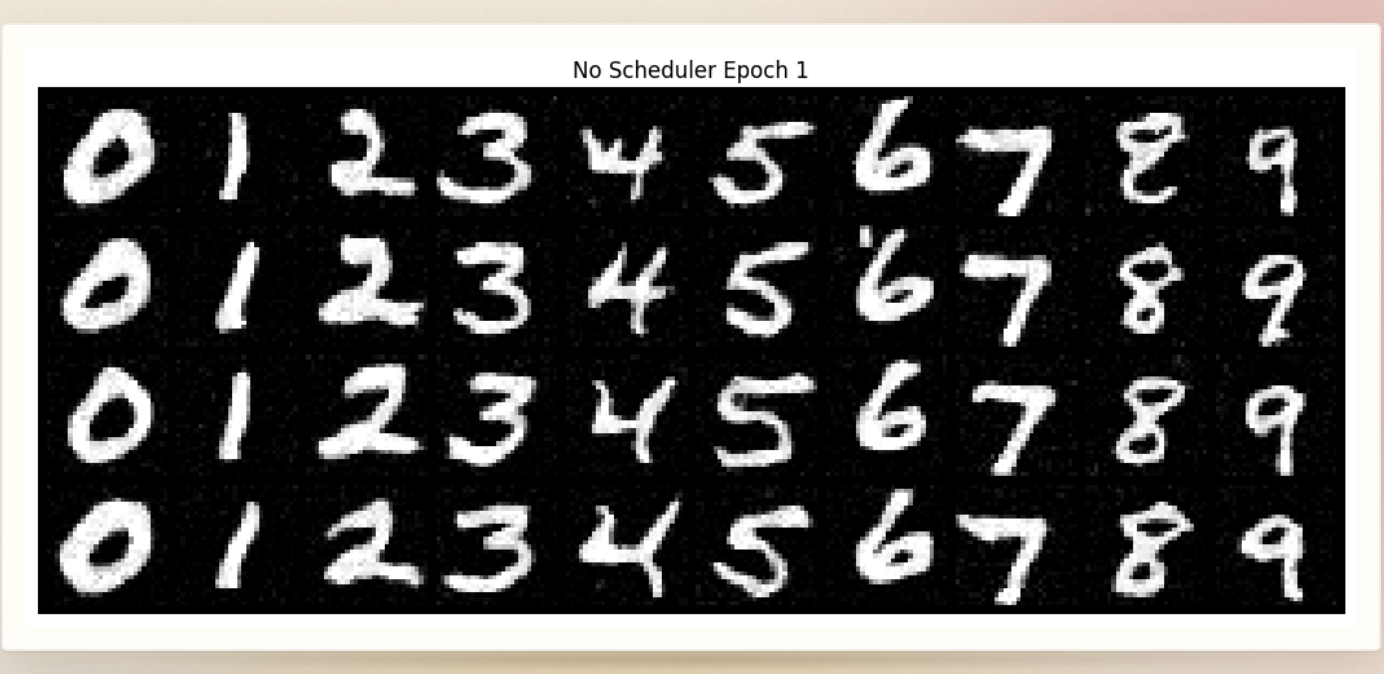
I noticed the scheduler's learning rate already decays towards 1e-3, so I decided that would be a good constant learning rate. It converges pretty well, and although the learning jumps around a bit more, for the most part it works.



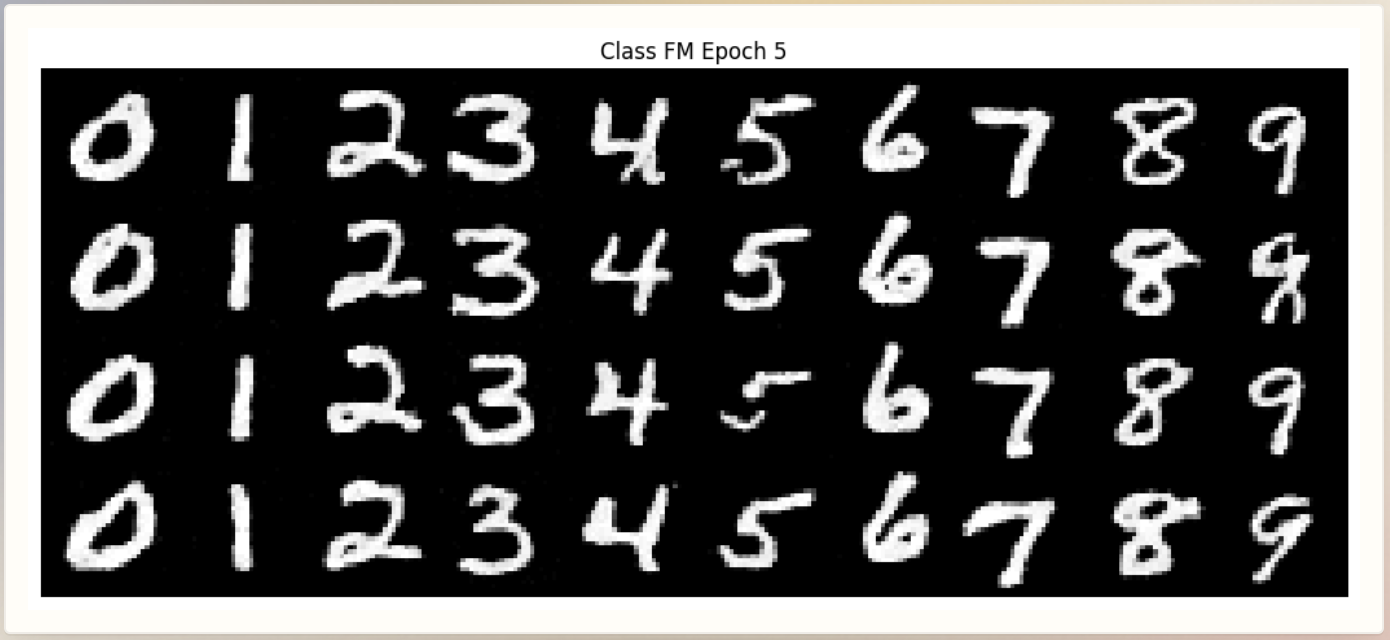
class-conditioned loss, no scheduler



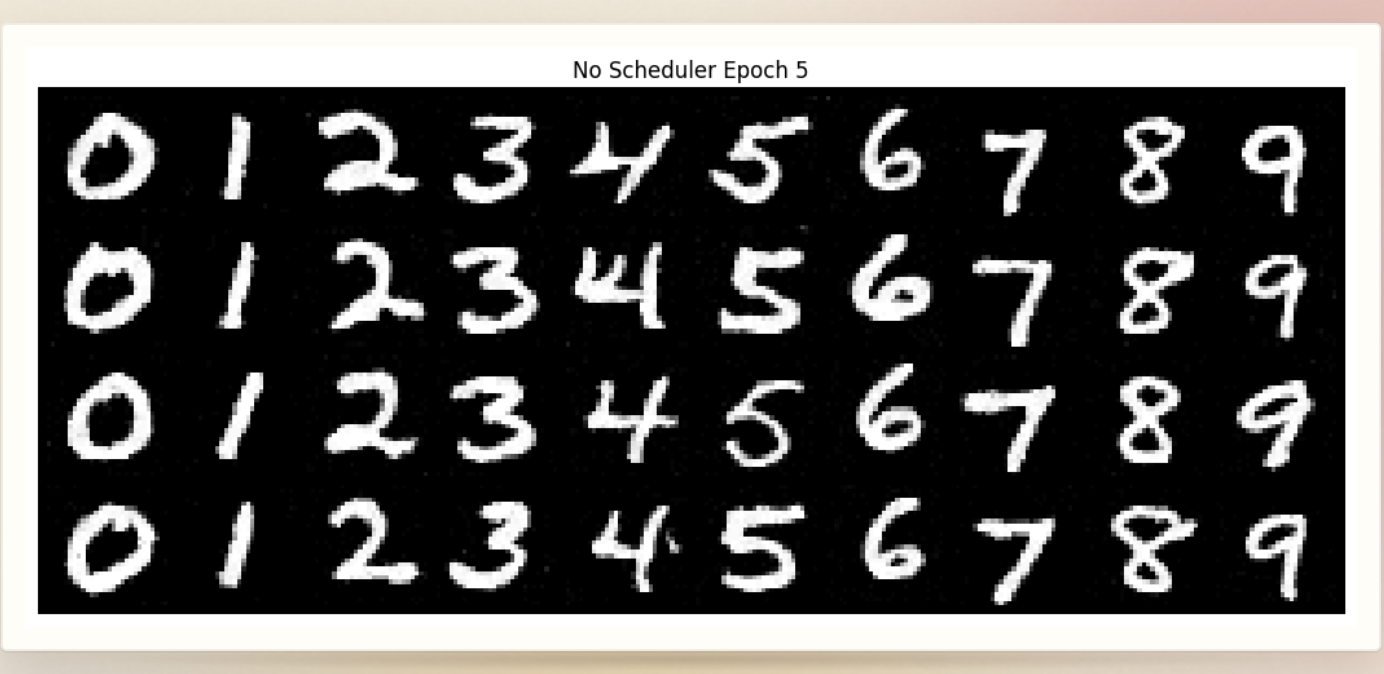
scheduler · epoch 1



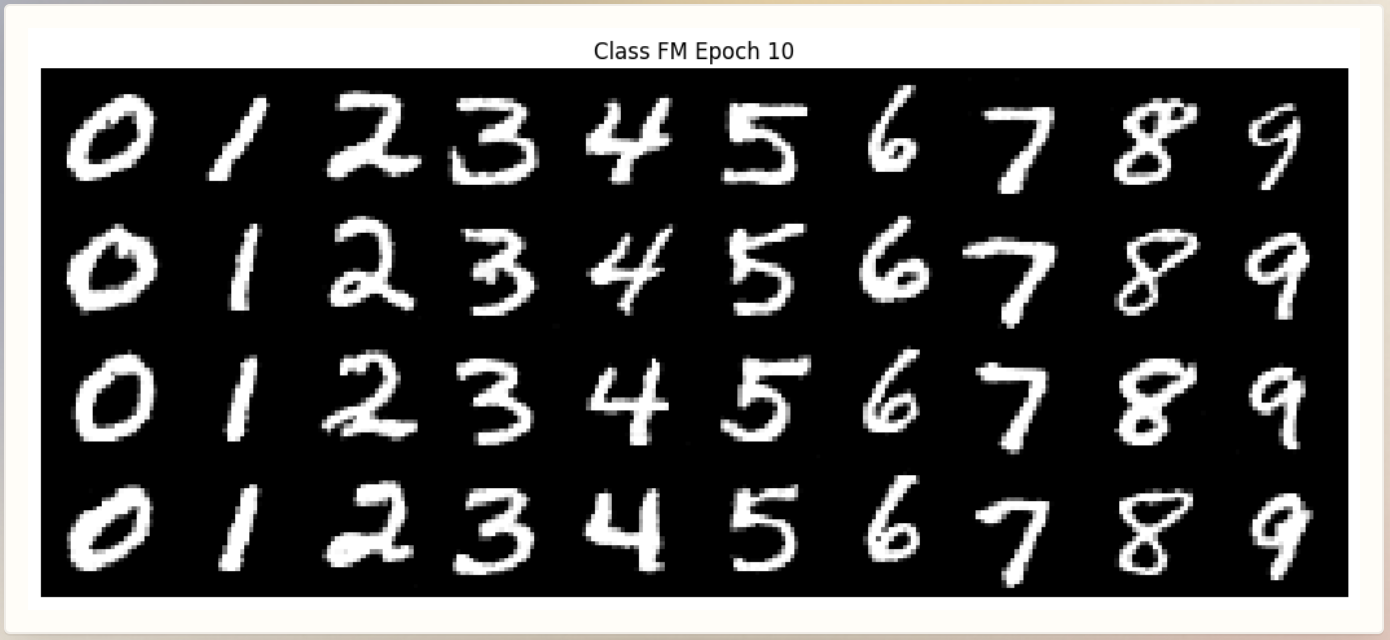
no scheduler · epoch 1



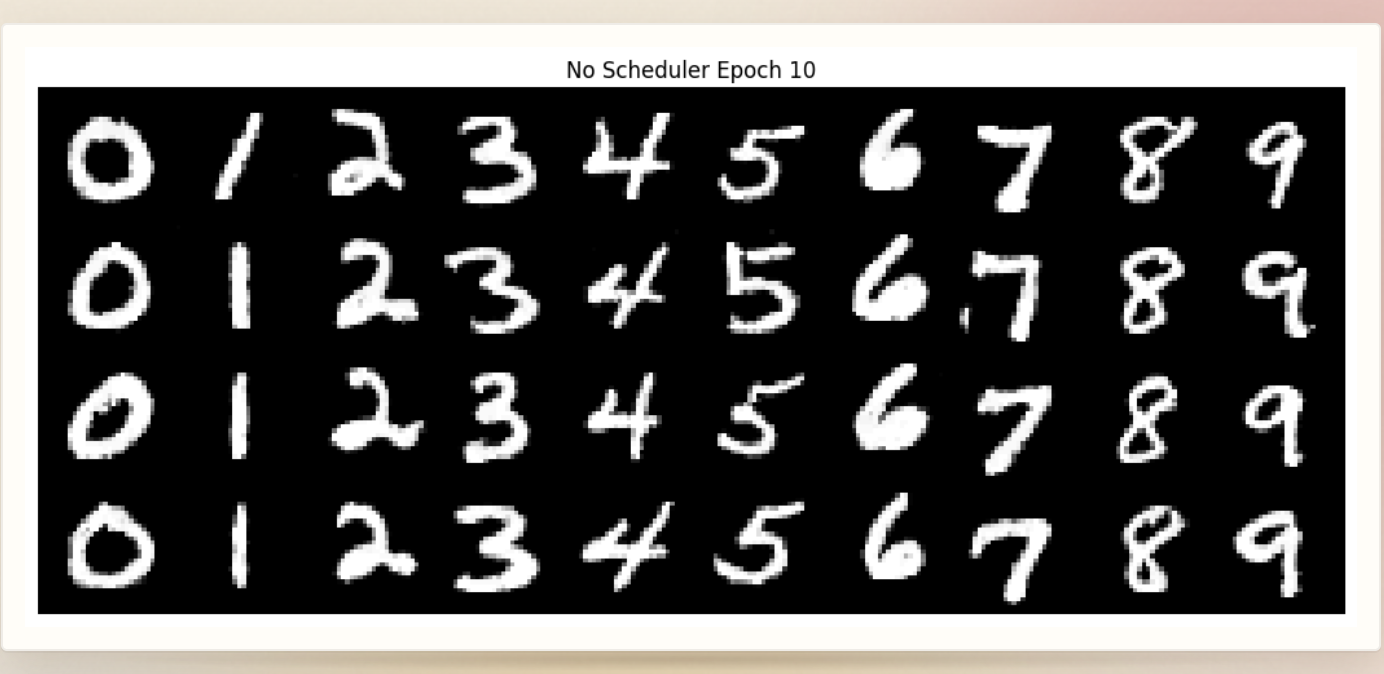
scheduler · epoch 5



no scheduler · epoch 5



scheduler · epoch 10



no scheduler · epoch 10